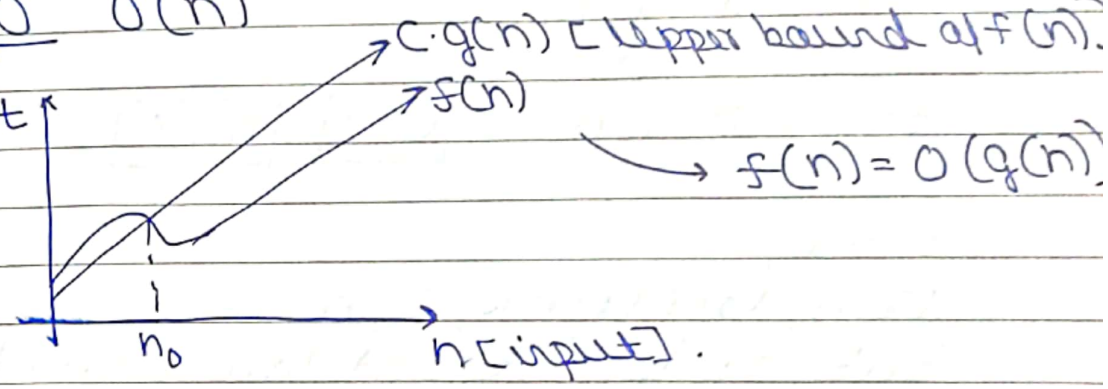
DS - ALGORITHM

* Time & Space Complexity

* Intro to Asymptotic Notations

- Algo → informal desc of Program
- choose the algo in terms of time & memory & write the prog.

1) Big O O(n)



$f(n) = O(g(n))$

$f(n) \leq c \cdot g(n) \quad ; \quad n \geq n_0$ where $c > 0, n_0 \geq 1$

eg: $f(n) = 3n + 2$.

$g(n) = n$.

$f(n) = O(g(n))$ ✓ g(n) bounds f(n).

$3n + 2 \leq c(n)$.

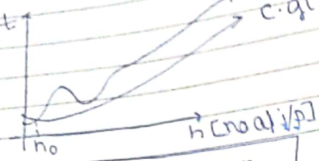
$\therefore 3n + 2 \leq 4n$. $c = 4$ $\therefore$ $n_0 = 2$ $c = 4$

$n \geq 2$

Big O → Tightest UB.

$n, n^2, n^3, \ldots, n^4$ } all true, but go for n

2) Big Omega (Ω)

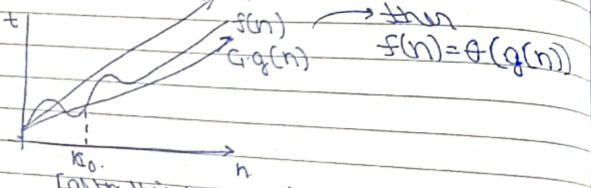


$f(n) \geq c \cdot g(n)$ for $n \geq n_0$
where $c > 0$, $n_0 \geq 1$

eg: $f(n) = 3n+2$, $g(n) = n$
$f(n) = \Omega g(n)$
$f(n) \geq c\,g(n)$
$3n+2 \geq cn$ $\quad c = 1$, $n_0 \geq 1$

$3n+2$,
$\log n$, $\log(\log n)$, 1, n ✓
But, go for Tightest lower Bound.

3) Theta (Θ)



then $f(n) = \Theta(g(n))$

[after this, both will agree]

$f(n) = \Theta(g(n))$
$c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$
where $c_1, c_2 > 0$, $n_0 \geq 1$

eg: $f(n) = 3n+2$, $g(n) = n$
$f(n) \leq c g(n)$ ✓
$f(n) = O(g(n))$ ✓
$f(n) = \Omega(g(n))$ ✓

- $\Theta \rightarrow$ asymptotically eq
  → $3n^2+n+1 \rightarrow \Theta(n^2)$

- O, Ω, Θ
  Given any algo → vibrated here
  ✓ $O \rightarrow$ Worst Case
  $\Omega \rightarrow$ Best Case
  ✓ $\Theta \rightarrow$ Avg Case → when best & worst case are same, we go for Θ.

Algo → Linear Search
$O \rightarrow O(n)$ [Worst case]
$\Omega \rightarrow \Omega(1)$ [Best case]
Avg case (Amortized Analysis) → $\Theta(n/2) = \Theta(n)$
(In this case, Θ done; but)

* Time Comp. of Iterative progs

- How to find $T(f(n))$? → count no of times loop exec

Algo → Iterative
→ Recursive. [func calling itself]
↳ in recursive funct

- (Both are equivalent in power)
  → if Algo is not recursive/iterative, it doesn't depend on i/p, ∴ constant time: $O(1)$

Scanned with CamScanner

* Iterative

① for(i=1 to n) → O(n)

② for(i=1 to n)
  for(j=1 to n) → O(n²)
  { }

③ A()
{ i=1, S=1;
  while(S<=n)
  { i++;
    S=S+i;
    pf("ravi");
  }
}

S=1, S=3, S=6, S=10, S=15, S=21 ✓

S	1	3	6	10	15	21	....	n
i	1	2	3	4	5	6	....	K.

loop run

$n = \frac{K(K+1)}{2}$

$K = O(\sqrt{n})$ ✓

∴ $\Theta(\sqrt{n})$

④ for(i=1; i²<=n; i++) → O(√n), Ans
A()
{ for(i=1; i<=n; i++) → n
  { for(j=1; j<=i; j++) → dep on i
    { for(K=1; K<=100; K++) → 100
      { pf }
    }
  }
}

i=1 j=1 time, K=100 times
i=2, j=2t, K=200t
i=3, j=3t, K=300t
.
i=n, j=nt, K=n×100 times

∴ 100 + 200 + 300 + ... n×100
⇒ $100 \cdot \frac{n(n+1)}{2}$
⇒ $O(n^2)$ ✓

⑤ for(i=1; i<=n; i++) → n. [Consoll it]
② { for(j=1; j<=i²; j++) → dep on i
    { for(K=1; K<=n/2; K++) → dep on n/2
      { pf() }
    }
}

i=1; j=1 time; K=n/2 times.
i=2; j=4t; K=4·n/2 t.
i=3; j=9t; K=9·n/2 t.
.
i=n; j=n²t; K=n²·n/2 t.

∴ $\frac{n}{2}[1+4+9+16+\cdots n^2]$

$= \frac{n}{2}\left[\frac{n(n+1)(2n+1)}{6}\right] = O(n^4)$. Ans ✓

Scanned with CamScanner

Q for(i=1; i<n; i=i*2) → O(logn)

i: 1, 2, 4, 8, 16 ··· n
1 2 3 K.

$2^{K-1} = n$

$K = \log n + 1$ ∴ $K = \log n$

Q for(i=n/2; i<=n; i++) → n/2
for(j=1; j<=n/2; j++) → n/2
for(K=1; K<=n; K=K*2) → logn
{ ps }

∴ $O\left(\frac{n}{2}\cdot\frac{n}{2}\log n\right) \Rightarrow O(n^2 \log n)$

Q for(i=n/2; i<=n; i++) — n/2
for(j=1; j<=n; j=2*j) — logn
for(K=1; K<=n; K=K*2) — logn
pf

∴ $O(n/2(\log n)^2)$

∴ $O(n(\log n)^2)$

Q A()
{ while(n>1)
{ n=n/2 }
}

n=2 — 1 t
n=4 — 2 t
:
n — logn times.

∴ $O(\log_2 n)$. Ans

Q for(i=1; i<=n; i++) — n
for(j=1; j<=n; j=j+i) — dep on i. (control)
pf()

i=1, j → n times.
i=2, j → n/2 times.
i=3, j → n/3 times.
:
i=n, j → 1 time.

∴ $n + \frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \cdots 1$

$= n\left[1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots \frac{1}{n}\right]$

∴ $O(n \log n)$

Q A()
{ n=2^2K
② for(i=1; i<n; i++) → n
{ j=2
while(j<=n) →
{ j=j^2
pf()
}
}

K=1; n=4; 2
K=2, n=16; 3
K=3, n=64, 4
K → , (K+1) time

$n = 2^{2K}$.

$\log n = 2K$.

$K = \log(\log n)$

∴ $O(\log\log n + 1)n$

∴ $O(n \log\log n)$

Scanned with CamScanner

* Time Analysis of Recursive Progs

```
A(n)
{
  if(n>1)                          — 1
  ret(A(n/2)+A(n/2))               —
}
```

$$T(n) = c + 2T\left(\frac{n}{2}\right)$$

Q
```
A(n)
{ if(n>1)
    return A(n-1)
}
```

$$T(n) = \underbrace{T(n-1)}_{1} + 1 \quad ; n>1$$
$$\quad ; n=1 \quad \text{(Base cond')}$$

① Back Substitution [Slow]

$T(n-1) = T(n-2) + 1$
$T(n-2) = T(n-3) + 1$

$T(n) = 1 + 1 + T(n-2)$
$\Rightarrow 2 + T(n-2)$
$\Rightarrow 2 + 1 + T(n-3)$
$= 3 + T(n-3)$

$K + T(n-K)$ → $n-K=1$, $K=n-1$
$(n-1) + T(n-(n-1))$ → $T(1)=1$, $K=n-1$
$\Rightarrow (n-1) + T(1) =$
n

$T(n) = O(n)$

Sum of n nos.

$$T(n) = \begin{cases} n + T(n-1) & ; n>1 \\ \Rightarrow 1 & ; n=1 \end{cases}$$

Soln $T(n-1) = n-1 + T(n-2)$
$T(n-2) = n-2 + T(n-3)$
⋮

$T(n) = n + n-1 + n-2 + T(n-3)$
$= 3n-3 + T(n-3)$
$\Rightarrow K(n-1) + T(n-K)$ → $T(1)=1$, $n-K=1$, $K=n-1$

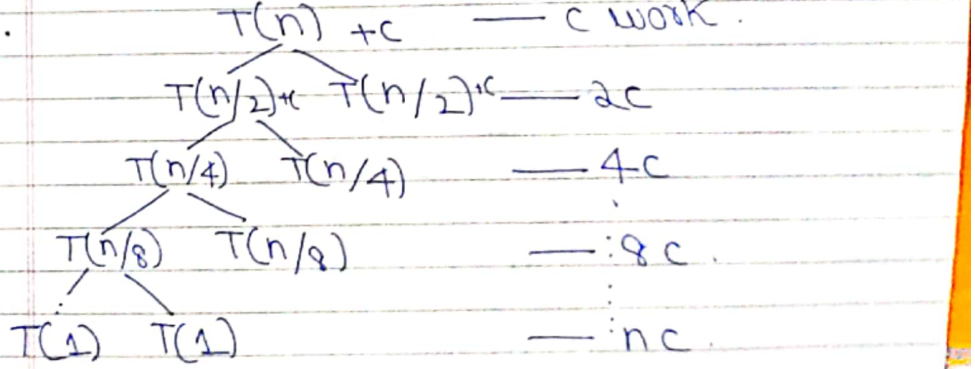
$\Rightarrow (n-1)(n-1) + 1$
$T(n) \Rightarrow n^2$
$\therefore T(n) = O(n^2)$

---

② Recursion Tree Method → some probs can be solved (not all)

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + c & ; n>1 \\ c & ; n=1 \end{cases}$$

[work done/level]



Scanned with CamScanner

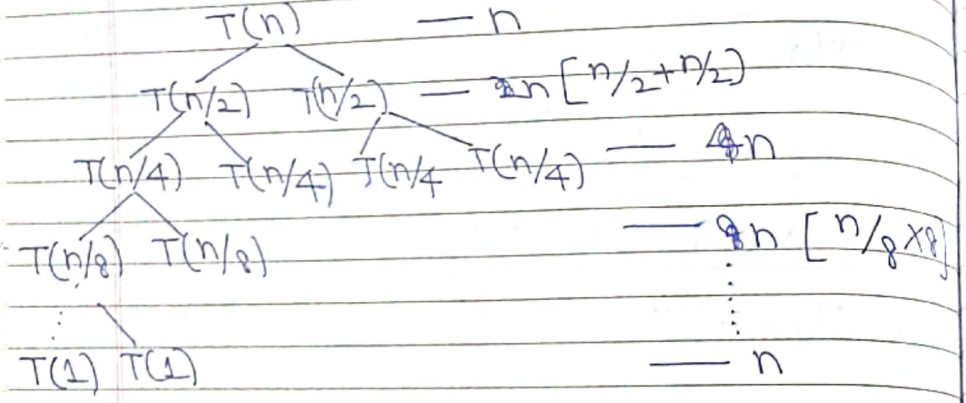
$\therefore c[1+2+4+8+\cdots n]$ → 2^K (assume) ⇒ GP.

$\Rightarrow c\left[\frac{1(2^{K+1}-1)}{2-1}\right]$ ∈ $\frac{a(r^n-1)}{r-1}$

$\Rightarrow c(2^{K+1}-1)$

$\Rightarrow c[2n-1] \rightarrow O(n)$

Q $T(n) = 2T(n/2) + n \; ; \; n>1$ → Merge Sort
$= 1 \; ; \; n=1$



∴ Work / level = n
No. of levels = log n

$n/2^2 = n/4$ — 2nd level
$n/8$ — 3rd level
n/n — log n level.

$\therefore T = O(n \log n)$

* Comparing Various functions to Analyse Time Complexity

Q $n^2 < n^3$

$n^2 \quad n^2 \cdot n$ [Scratch out comm terms]
$1 < n$.

→ Substitute Very large Value of n, & comp.
→ log both sides.
→ Constants dont matter in asymptotically.

Q $2^n \quad n^2$.
$n \log_2 2 \quad 2 \log_2 n$.

$\Rightarrow n > 2 \log n$ ← Sub $n = 2^{100}$.

$2^{100} > 2 \times 100 = 200$

Q $3^n \quad 2^n$.
$n \log 3 \quad n \log 2$
$\log 3 > 1$

Q $n^2 \quad n \log n$. → $O(n^2)$
$n > \log n$.

Q $n \quad (\log n)^{100}$. → $O(n)$

$\Rightarrow \log n \; , \; 100 \cdot \log(\log n)$
$\Rightarrow 128 \; , \; 700$.
$\Rightarrow 1024 \; , \; 1000$.
$= 2^{300} > 300,00$

$n = 2^{2^7}$
$n = 2^{2^{70}}$.
$2^{2^{300}}$

Scanned with CamScanner

Q $n^{\log n}$ $n \log n$

$\log n \cdot \log n$, $\log n + \log(\log n)$

$\Rightarrow$ $(1024)^2$, $1024 + 10$. $n = 2^{1024}$

$\Rightarrow$ $2^{20} \cdot 2^{20} > 2^{20} + 20$. $n = 2^{2^{20}}$

Q $\sqrt{\log n} > \log \log n$.

$\Rightarrow \frac{1}{2}\log(\log n)$ $\log(\log(\log n))$

$\Rightarrow 5$, $\log 10 \approx 3$. $n = 2^{2^{10}}$

$n =$

Q $n\sqrt{n} > n \log n$

$\sqrt{n} \log n$ $\log n \cdot \log n$.

$\Rightarrow \sqrt{n}$, $\log n$.

$\Rightarrow \frac{1}{2}\log n$ $\log \log n$. $n = 2^{2^{10}}$.

$\Rightarrow 512 > 10$

Q $f(n) = \begin{cases} n^3 , & 0 < n < 10,000 \\ n^2 , & n \geq 10000 \end{cases}$

$g(n) = \begin{cases} n , & 0 < n < 100 \\ n^3 , & n > 100 \end{cases}$

- we are intrested in very large values.

$n^2 < n^3$.

	0-99	100-9999	10,000-∞
$f(n)$	n^3	n^3	n^2
$g(n)$	n	n^3	n^3

$\therefore$ $g(n) > f(n)$ $f(n) = O(g(n))$

$n_0 = 10K$.

Q $f_1 = 2^n$ ex , $f_2 = n^{3/2}$ p , $f_3 = n \log n$ p , $f_4 = n^{\log n}$ ex

$2^n > n^{3/2}$	$2^n > n \log n$.	$2^n > n^{\log}$
n , $\frac{3}{2}\log n$	n , $\log n + \log \log n$	n , $(\log n)$
2^{128} , 3×64	2^{128} , $128 + 7 = 135$.	2^{128} , $(128)^2$

$\therefore$ 2^n is largest (1) f_1

$n^{\log n} > n^{3/2}$ $n^{\log n} > n \log n$.

$\Rightarrow \log n \log n$, $\frac{3}{2}\log n$. $\log n \log n$, $\log n + \log \log$

$(128)^2$, $128 + 7$

$n^{\log n}$ — (2) f_4

$n^{3/2} > n \log n$ $\therefore$ $n^{3/2}$ — (3) f_2

$\Rightarrow \frac{3}{2}\log n$, $\log n + \log \log n$ $n \log n$ — (4)

$\Rightarrow \frac{3}{2} \cdot 128$, $128 + 7 = 135$

$\frac{3}{2} \cdot 1024$ $1024 + 10 = 1034$

Scanned with CamScanner

* Master's Theorem

used to solve recurrence eqn's

$T(n) = aT(n/b) + \theta(n^k \log^p n)$

$a \geq 1, b \geq 1, k \geq 0$ & p is real no.

1) if $a > b^k$, then $T(n) = \theta(n^{\log_b a})$

2) if $a = b^k$
   a) if $p > -1$, then $T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
   b) if $p = -1$, then $T(n) = \theta(n^{\log_b a} \log\log n)$
   c) if $p < -1$, then $T(n) = \theta(n^{\log_b a})$

3) if $a < b^k$
   a) if $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$
   b) if $p < 0$, then $T(n) = O(n^k)$

* $(\log n)^2 \| \log^2 n \rightarrow \log\log n$ (imp)
⇒ $\log^p n \rightarrow (\log\log\log \cdots n)$ (p times)

1. $T(n) = 3T(n/2) + n^2$
   $a=3, b=2, K=2, p=0$.
   3 < 4. $a < b^K$.
   $T(n) = \underline{\theta(n^2)}$

2. $T(n) = 4T(n/2) + n^2$
   $a=4, b=2, k=2, p=0$.
   $a = b^K$.
   $\therefore T(n) = \theta(n^{\log_2 4} \log n) = \underline{\theta(n^2 \log n)}$

3. $T(n) = T(n/2) + n^2$
   $a=1, b=2, K=2, p=0$.
   ⇒ $a < b^K$.
   ⇒ $T(n) = \underline{\theta(n^2)}$

4. $T(n) = 2^n T(n/2) + n^n$.
   $a = 2^n$ ✗ a needs to be const.

5. $T(n) = 16T(n/4) + n$
   $a=16, b=4, K=1, p=0$
   $a > b^K$.
   ⇒ $T(n) = \theta(n^{\log_4 16}) \Rightarrow \underline{\theta(n^2)}$

6. $T(n) = 2T(n/2) + n\log n$
   $a=2, b=2, K=1, p=1$.
   $a = b^K$.
   $\therefore T(n) = \underline{\theta(n \log^2 n)} \rightarrow \underline{\theta(n \log\log n)}$.

7. $T(n) = 2T(n/2) + n/\log n$
   ⇒ $2T(n/2) + n\log^{-1} n$.
   $\therefore a=2, b=2, K=1, p=-1$
   $a = b^K$.
   $T(n) = \theta(n^{\log_2 2} \log\log n)$
   ⇒ $\underline{\theta(n \log\log n)}$

8. $T(n) = 2T(n/4) + n^{0.51}$.
   $a=2, b=4, K=0.51, p=0$.
   $\therefore a < b^K$.
   ⇒ $T(n) = \underline{\theta(n^{0.51})}$.

Scanned with CamScanner

9) $T(n) = 0.5T(n/2) + 1/n$ X
$a = 0.5$

10) $T(n) = 6T(n/3) + n^2 \log n$
$a=6, b=3, K=2, p=1$
$a < b^K$
$\Rightarrow T(n) = \theta(n^2 \log n)$

11) $T(n) = 64T(n/8) - n^2 \log n$ X

12) $T(n) = 7T(n/3) + n^2$
$a=7, b=3, K=2, p=0$
$a < b^K$
$T(n) = \theta(n^2)$

13) $T(n) = 4T(n/2) + \log n$
$a=4, b=2, K=0, p=1$
$a > b^K$
$T(n) = \theta(n^{\log_2 4}) = \theta(n^2)$

14) $T(n) = \sqrt{2}\,T(n/2) + \log n$
$a=\sqrt{2}, b=2, K=0, p=1$
$a > b^K$
$\Rightarrow \theta(n^{\log_2 \sqrt{2}}) = n^{1/\log_2 2^{1/2}}$
$\theta(\sqrt{n})$

15) $T(n) = 2T(n/2) + \sqrt{n}$
$a=2, b=2, K=1/2, p=0$
$a > b^K$
$\therefore \theta(n)$

16) $T(n) = 3T(n/2) + n$
$a=3, b=2, K=1, p=0$
$a > b^K$
$\therefore \theta(n^{\log_2 3})$

17) $T(n) = 3T(n/3) + \sqrt{n}$
$a=3, b=3, K=1/2, p=0$
$a > b^K$
$\therefore \theta(n^{\log_3 3}) = \theta(n)$

18) $T(n) = 4T(n/2) + cn$
$a=4, b=2, K=1, p=0$
$a > b^K$
$\therefore \theta(n^2)$

19) $T(n) = 3T(n/4) + n \log n$
$a=3, b=4, K=1, p=1$
$\therefore T(n) = \theta(n \log n)$

* Analyse Space Complexity
→ how many mem cells required
→ how much max extra space, depending on size of I/p

* Iterative →
```
Algo(A, 1, n)
{ int i;
  for(i to n)
    A[i] = 0
}
```
→ Space = O(1) (doesn't depend on I/p)

Scanned with CamScanner

Q Algo(A,1,n)
{ int i;
create B[n];
for(i=1 to n)
B[i]=A[i]
}
→ Space = O(n)

Q Algo(A,1,n)
{
create B[n][n];
int i,j
for// fill B with A
}
→ Space = O(n²)

* Recursive Prog

Head
Tail
Body
} rec.

→ A(n)
{ if(n≥1) → base condⁿ (n<1)
{ A(n-1) → Head Recursion (downward)
pf(n);
} }

(Tree Method (if prog is small))

A(3)
A(2) P(3)
A(1) P(2)
A(0) P(1)

→ 4 recursive calls.
∴ (n+1) calls.

OS stack.

A(0)
A(1)
A(2)
A(3)

1 2 3

Space = depth of Stack
⇒ O(n)

A(0) is done, now it is killed, pop it.

Time complexity

T(n) = T(n-1) + 1
(Back sub)
T(n-1) = T(n-2) + 1
T(n-2) = T(n-3) + 1

T(n) = T(n-3) + 3
T(n) ⇒ T(n-K) + K. n-K=1, K=n-1
T(n) ⇒ T(1) + n-1
∴ T(n) = n ∴ T(n) = O(n)

Q A(n)
{ if(n≥1) Tree Method →
{ A(n-1);
pf(n);
A(n-1);
} }

A(3)
A(2) P(3) A(2)
A(1) P(2) A(1) A(1) P(2)
A(0) P(1) A(0) A(0) P(1) A(0)

→ 15 function calls.

A(3) → 15
A(2) → 7
A(1) → 3

n → 2^(n+1) - 1

1) ∴ T = O(2^n) → no of recursive calls

2)

A(0)	A(0)
A(1)	A(1)
A(2)	
A(3)	

depth of Stack = depth of Tree
⇒ O(n+1)
∴ S = O(n) → dep of T

Scanned with CamScanner

O/P : 1 2 1 (3) 1 2 1
A(2) A(2)

Save results of A(2), A(1), A(0)
[Use DP/Memoization] → T=O(n) (DP)

0 1 2 3
{} {1} {121} A(2) 3 A(2)

If Subproblems are repeating, Overlapping Subproblems

· Time Complexity
$T(n) = 2T(n-1) + 1$ $n=0 \rightarrow 1$
(Back Substn)
$T(n-1) = 2T(n-2) + 1$
$T(n-2) = 2T(n-3) + 1$

$T(n) = 2\,2\,2\,T(n-3) + 3$
$\Rightarrow 2^3 T(n-3) + 3$

$T(n) = 2^K T(n-K) + 2^{n-1} + 2^{n-2} \ldots 1$ $n-K=0$
$\Rightarrow 2^n T(0) + n$ (GP) $K=n$
$\Rightarrow 2^n + n$
$T = O(2^n)$

$T = O(2^n)$
$S = O(n)$

With DP

$T = O(n)$
$S = O(n)$

* Amortized Analysis

- Instead of compute cost per opn, find avg cost (of series of operations) (Sequence of DS operations)
- avg cost → small.
- even though single opn → expensive
eg Hash Table, Splay Tree, Disjoint Sets.

Techniques → Aggregate
→ Accounting
→ Potential

* Aggregate Analysis

$\frac{T_1 + T_2 + \cdots T_n}{n}$ → time for nth opn

eg: Hash Table

- if full
- Allocate twice memory & copy
- free old table

→ Insertion: (diff or diff cases) → 1+1+1 ... n+1+1

* Example on Aggregate Analysis

Item no:	1	2	3	4	5	6	7	8	9	10
Table Size:	1	2	4	4	8	8	8	8	16	16
Cost:	1	2	3	1	5	1	1	1	9	1

2 moves, 1 insertn.

8 moves, 1 insert

Scanned with CamScanner

Normal Analysis

- Worst case: → O(n) [full/double]
  O(n²) (for all ops n)

- The Amortized cost = $\frac{1+2+3+1+5+6+1+1+1+9}{n}$ (9 ops, n)

$\Rightarrow \frac{(\overbrace{1+1+\cdots 1}^{n}) + (1+2+4\cdots 8)}{n}$

$\Rightarrow \frac{n + \log_2 n}{n}$

$\Rightarrow O(1)$

$\frac{a(r^n-1)}{r-1}$

$\Rightarrow \frac{1(2^{\log n}-1)}{1}$

Go with aggregate Analysis to find TUB.

* Sorting Techniques

* Insertion Sort Algo

- Playing cards (All cards downwards, pick & compare with left hand cards).
- involves shifting.

$1+2+3+\cdots\cdots n-1$.

$\Rightarrow O(n^2)$ [worst case → Rev. Sorted]

Best case → Sorted Order

[O(n) comparisons]

Go find loc to insert (decrease comparison)

Space Complexity: O(3) = O(1)

Time → [no of comp → if BS: O(log n)
no of movements: O(n).]

1)

∴ even if binarySearch → O(n²)

2) * To decrease movements → double LL.

Comp: O(n) (to find loc).
movement: O(1) [change links only]

9 ⇄ 5 ⇄ 1 ⇄ 2 ⇄ 4

∴ even with LL → O(n²).

* Merge Sort Algorithm → good time.

* Merge

0	1	2	3	4	5	6	7
1	5	7	8	2	4	6	9
p			q	q+1			r

2 Sub Array

L: 1 5 7 8 | ∞ ∞ ∞ ∞ | size: q−p+1
i

R: 2 4 6 9 | ∞ ∞ ∞ ∞ | size: r−q
j

1	2	4	5	6	7	8	9
K							

[Original list written]

Scanned with CamScanner

- Copy: O(n) [into a list]
- (n+n) [Compare & copy to og Arr]

O(2n)

Q To Merge O(n+m)

Space = O(n+m)
Time = O(n+m)

* Out of Place Procedure

* Why ∞ added at the end?

A [20 20 30 40 1 5 6 9] (p, q, q+1, r)

L [20 20 30 40 ∞]

R [1 5 6 9 ∞]

A [1 5 6 9 10 20 30 40]

∞: if one list is over, other list is copied completely

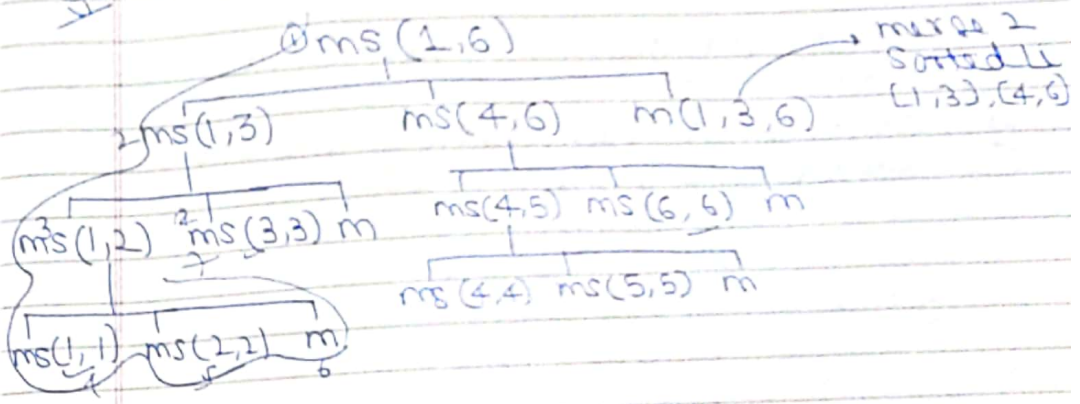
if n → Space = O(n), Time = O(n)

* Merge Sort → Out of Place Algorithm

Divide & Conquer.

- Divide prob into many no: of SP, solve those SP just, & then combine the solution.

Q A [9 6 5 0 8 2] (1 2 3 4 5 6)



→ divide the array, till we get a single no.

* Space Complexity → O(n+log n) ⇒ O(n)

merge → O(n)
mergeSort (recursion)

- Ht of Stack = depth of tree (no of levels)

→ each cell in Stack = all elements of some level

∴ n → (⌈log n⌉ + 1) levels

Size of Stack = O(log n) + n [Merge]

Scanned with CamScanner

* Time complexity

$T(n) = 2T(n/2) + n$
2 MS calls Merge

$\Rightarrow \theta(n \log n)$ [Best = Worst = Avg]

Q Given n elements, merge them into sorted list using Merge Sort.

∴ No of levels $= \log_2 n$ (∵ height, 3 levels)

∴ Work/level $= n$.

$T = O(n \log n)$

Q Given $\log n$ sorted lists, each of size $n/\log n$ whose n the total size, required to merge them into single list.

n/logn
n/logn
n/logn

1) MS uses Divide & Conquer Paradigm
2) O(m+n)
3) 20 47 15 8 9 4 40 30 12 17

Pass 1: 20,47 | 8,15 | 4,9 | 30,40 | 12,17

Pass 2: 8,15,20,47 | 4,9,30,40 | 12,17

→ Elements after Pass 2 [2 Way Merging]

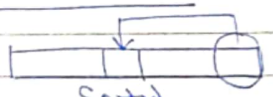
* Quick Sort Algorithm → DnC Prob

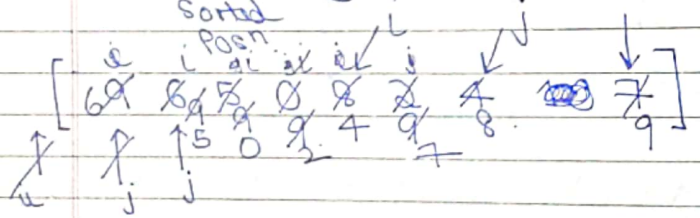
Heart → Partition Procedure [Common]

: if size of i/p is small, QS better than MS

* Partition [Put 1 correct in Sorted Pos] i=-1, j=0, pivot = last

(choosing last elem as pivot)





if $j \le$ pivot → inc i & swap i, j.

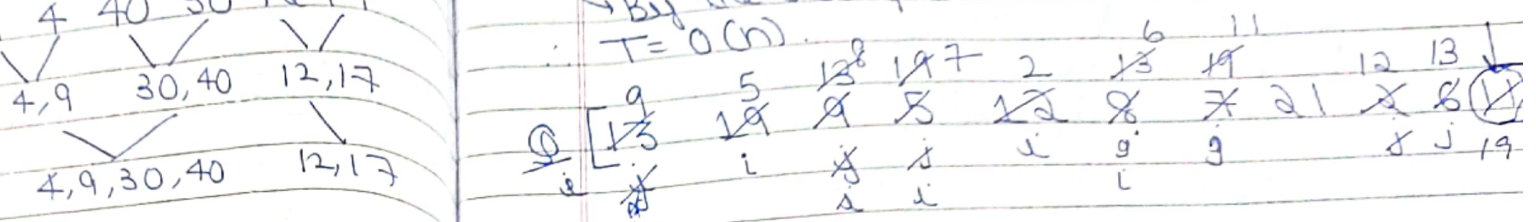
(all elem less than 7, are in start of list)

exchange i+1 with 7

[6 5 0 2 4 7 8 9]

→ By the end of 1 Partition

∴ T = O(n)



[9 5 8 7 2 6 11 21 12 13 19]

<= >

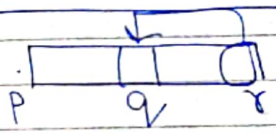
T = θ(n) → Sorted / UnSorted

∴ j runs from start to end.

• Partition Algo → Call

* QS(A, p, r)

```
{ if (p < r)
  { q = Partition(A, p, r)
    QS(A, p, q-1)
    QS(A, q+1, r)
} }
```



dep upon, Partition idx. Complexity will vary. Subproblems depend.

Scanned with CamScanner

1 3 2 (4) 7 6 5.

1 2 3 (4) 7 6 5.

1 3 2 5 7 6 (4) $q \to 4$.

eg. [5 7 6 1 3 2 4]

qs(1,7) [5 6 7]

q_1 = P(1,7) qs(1,3) qs(5,7)

[1 2 3]

2 = P(1,3) qs(1,1) qs(3,3)

⇒ Space Complexity

No. of Stack records = No. of levels = ht of Tree

→ Best Case : log n (Partn at mid)

→ Worst Case : n (Partn at end)

∴ Space Complexity

Θ(log n)

[Aggr. Analy]

ht = O(n)

n

1 n−2

1 n−4.

⇒ Time Complexity

- $T(n) = \underbrace{O(n)}_{\text{Partn}} + 2T(n/2)$ → [Partn at middle Best Case]

⇒ O(n log n) - Best Case

- $T(n) = n + 1 + T(n-1)$ → [Partn at last Worst Case]

⇒ $O(n^2)$ → Worst Case

* Analysis of Quick Sort & Problems on it

Q1 [1 2 3 4 5 (6)] (Asc)

Partn at last → Ht of Tree : O(n)

∴ $T = O(n^2)$

//Skewed Tree

Time = $O(n^2)$
Space = O(n)

$T(n) = O(n) + T(n-1) + O(0)$

$\Rightarrow O(n^2)$

Q2 [6 5 4 3 2 (1)] (Desc)

Partn at beginning → Ht of Tree : O(n)

$T = O(n^2)$ → $T(n) = O(n) + T(n-1)$

//Skewed Tree

Q3 [2 2 2 (2)] (All of elements)

Ht of Tree : O(n)

$T(n) = O(n) + T(n-1) = O(n^2)$

* n → Partn : O(n)

with only such Splits : $O(n^2)$

n

0 n−1

0 n−2

0 n−3

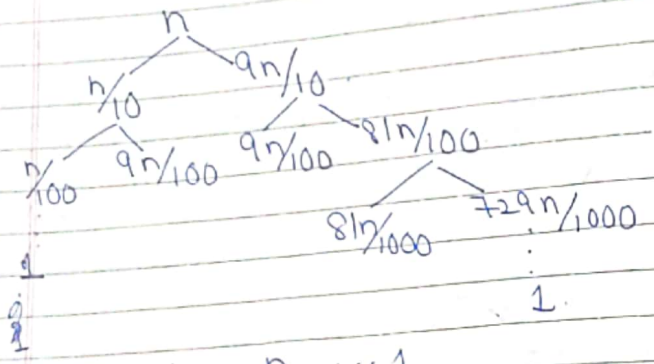
n × n = $O(n^2)$

Work/Level Level

Scanned with CamScanner

Observations

Q ✱ Split input into 1:9 [Work/level] — O(n)



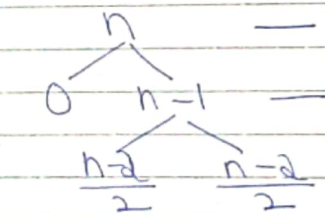
$n, \frac{n}{10/9}, \frac{n}{(10/9)^2} \cdots 1$

$\Rightarrow$ [No of Levels = $\log_{10/9} n$]

$T = O(n \log n)$ [Even if the split is 1:9]

1:99 1:999 1:9999 → $O(n \log n)$	(Any Ratio)

Q Alternate [But Worst]



— cn [Worst Split]

— cn [But Worst Split]

$T(n) = cn + cn + 2T\left(\frac{n-2}{2}\right)$

$\Rightarrow O(n) + 2T\left(\frac{n}{2}\right)$

$\Rightarrow n \log n$

Q Median of n elements → O(n)
Complexity of QS, if median is pivot.

1) First find median, O(n), (extra) put it in end of list
2) then Partn also O(n)

$\therefore T(n) = O(n) + O(n) + 2T(n/2)$

$T(n) \Rightarrow O(n \log n)$ ✓

Q The $(n/4)^{th}$ Smallest element is selected as pivot using O(n) time algo. Find the Worst Case time of QS

1:3

$T(n) = T(n/4) + T(3n/4) + O(n)$ Partitn.

$\Rightarrow T(n) = \Theta(n \log n)$

Q [1, 2, 3, 4, 5 ... n] → T_1
[n, n-1, n-2, ... 1] → T_2

$T_1 = T_2 = O(n^2)$

Q $T(n) = T(n/5) + T(4n/5) + O(n)$

$= \Theta(n \log n)$

Scanned with CamScanner

* Intro to Heaps

UP [CDS] → Algo

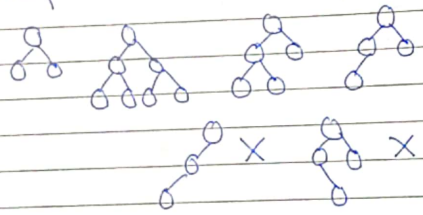
- The DS affects the Time Complexity of Algo.

	insert	search	findMin	DeleteMin
eg. Unsorted Array	O(1)	O(n)	O(n)	O(n)
Sorted Array	O(n)	O(logn)	O(1)	O(n)
LinkedList (UnSorted)	O(1)	O(n)	O(n)	O(n)
· min Heap	O(logn)	O(n)	O(1)	O(logn)

↳ optimized wrt findMin, deleteMin

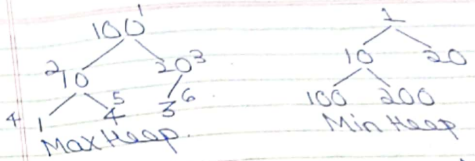
* Heap is a binary tree or 3-ary or n-ary tree

· Heap → almost complete Binary Tree



: Heap [no dummy nodes] [no Gaps in Array]

* Max Heap: Heap in which the root is always maximum; than Left & Right ST.



* Heap is implemented as an Array & visualize as Tree.

1	2	3	4	5	6
100	10	20	1	4	3

parent = p
left = 2p
right = 2p+1

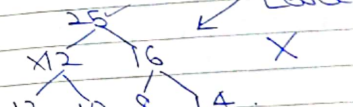
parent = ⌊n/2⌋

Parent(i) = ⌊i/2⌋
Left(i) = 2i
Right(i) = 2i+1

} Right Shift
} Left Shift open
↳ easy in comp.

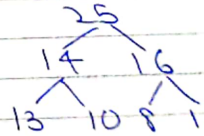
Q1 25, 12, 16, 13, 10, 8, 14. Is it Ma

Level Order



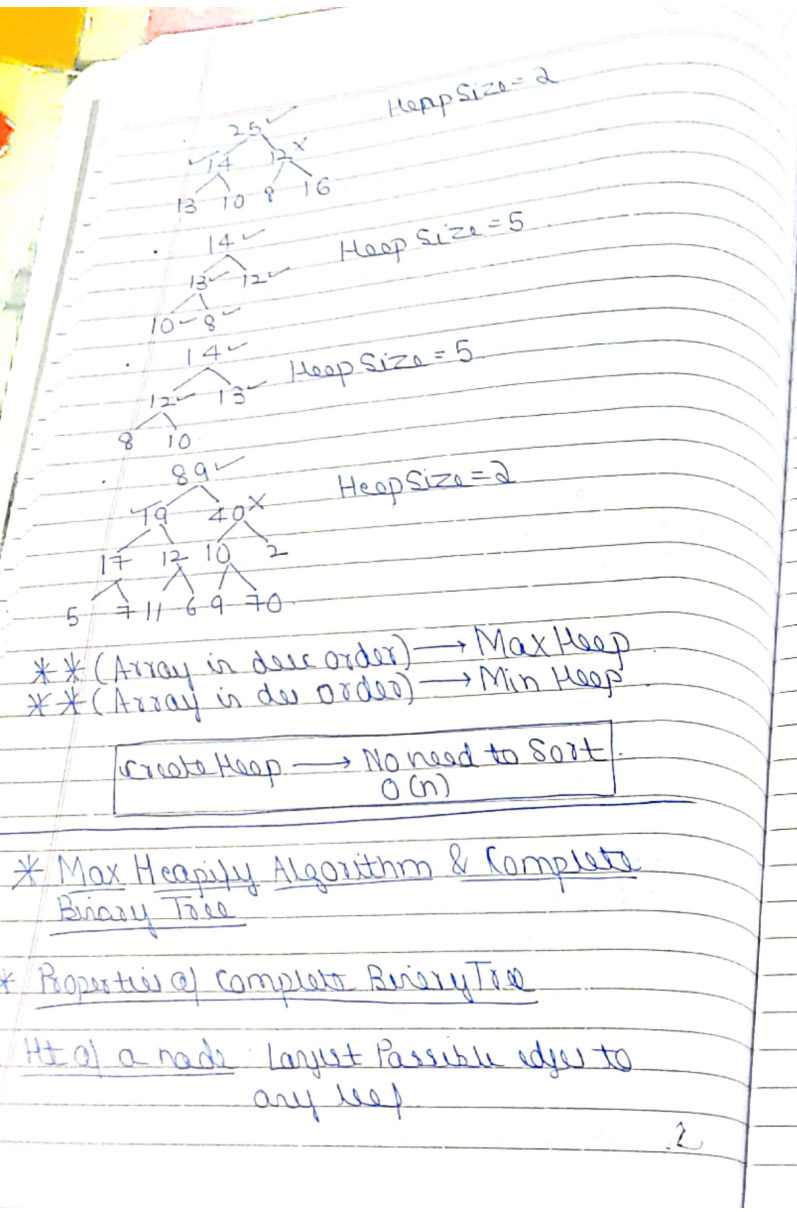
How many elements following Heap Property [Heap Size] → 1 [2 failed

Q 25, 14, 16, 13, 10, 8, 12



Heap Size = 7

Scanned with CamScanner



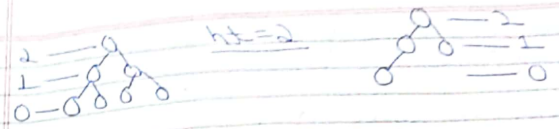
** (Array in desc order) ⟶ Max Heap
** (Array in asc order) ⟶ Min Heap

Create Heap ⟶ No need to Sort
O(n)

* Max Heapify Algorithm & Complete Binary Tree

* Properties of Complete Binary Tree

Ht of a node: Longest Possible edges to any leaf



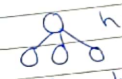
2) ht of a tree = ht of root

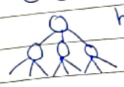
[ht of Tree = 2]

3) Given ht max no. of nodes (Complete B

$h \longrightarrow 2^{h+1} - 1$

* 4 Ternary Tree





$h = n \longrightarrow 1, 3, 9, \ldots 3^n$

$h \rightarrow \frac{1(3^{h+1} - 1)}{2}$

4) * if n nodes ⟶ (min ht) $\lceil \log_2(n+1) \rceil - 1$

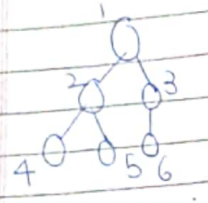
or

n nodes ⟶ $\lceil \log n \rceil$

Scanned with CamScanner

* Given an Array of elements, build a MaxHeap

1) Sort in desc order → O(nlogn)



Note: In a CBT, if leaves start they will go till end

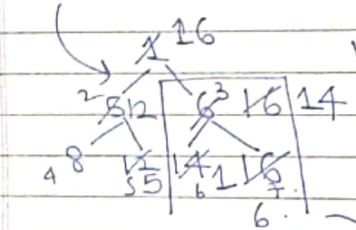
Prop of CBT: $\left\{\left\lfloor\frac{n}{2}\right\rfloor+1 \text{ to } n\right\}$ are leaves

4 to 6 are leaves

∴ every leaf is a Heap.

* Build Heap Algo (MaxHeap)

eg: [1 5 6 8 12 14 16]

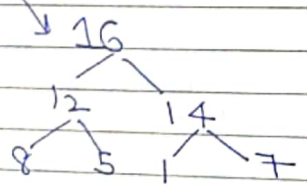


Leaf: $\left\lfloor\frac{n}{2}\right\rfloor+1$ to n.

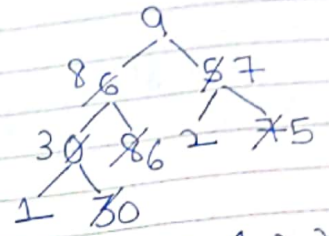
∴ 4 to 7: Leaf

1 to 3: NonLe

- Start from 3, 2, 1



eg: 9 6 5 0 8 2 7 1

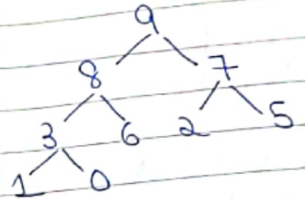


Leaf: [5 to 8]

: $\left\lfloor\frac{n}{2}\right\rfloor+1$ to n.

NonLeaf: [1 to 4]

- Start from 4, 3, 2, 1



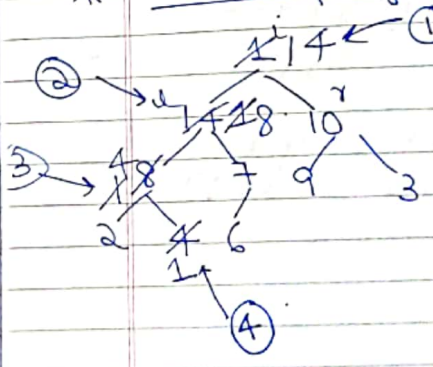
Algo (CreateH)

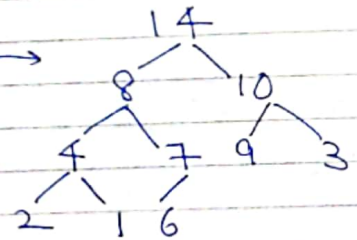
1) All leaves are maxHeap.
2) Start from largest index, which is NonLeaf & Apply Heapify there.

- If left & right ST is maxHeap, what to do with root, so as to make the tree: maxHeap.

[MaxHeapify]

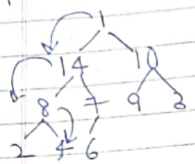
* MaxHeapify → Call





Scanned with CamScanner

4. L & R are max Heap, & root is not



Max diff, root = $\Theta(\log n)$ travel

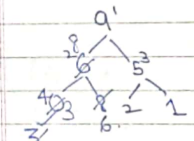
∴ MaxHeapify/MinHeapify

$T = O(\log n)$
$S = O(\log n)$ (recursion)

- MaxHeapify time depends on what level you are calling Heapify.
  if leaf: O(0)

* Build Max Heap

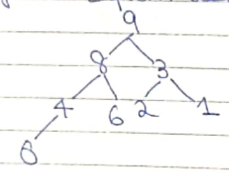
[9 6 5 0 8 2 1 3]



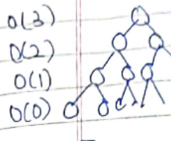
Leaf ∈ $[\frac{n}{2}]+1$ to n.

Non Leaf ∈ $[n/2]$ to 1 (down)

- from 4, 3, 2, 1 ; apply heapify and they are [from largest non leaf to 1]



- Time Complexity Analy (Build Heap)



- [No. of nodes at height h in Heap (Level) [max] → $\lceil \frac{n}{2^{h+1}} \rceil$] ← Heap

eg: $h=2 \longrightarrow \lceil \frac{15}{2^{2+1}} \rceil = 2$ nodes

- MaxHeapify → O(h)

∴ MaxHeapify on all nodes (Build Heap)

$$\sum_{h=0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) \Rightarrow \frac{cn}{2} \sum_{h=0}^{\log n} \left(\frac{h}{2^h}\right)$$

(nodes in a level of ht h)

$$\Rightarrow O\left(\frac{cn}{2} \underbrace{\sum_{h=0}^{\infty} \frac{h}{2^h}}_{2}\right)$$

$\therefore O(n)$

∴ Build Max Heap = O(n) (from a random array)

Space Complexity = $O(\log n)$ → Max Space taken by any path call
Time Complexity = $O(n)$

↑ Build Heap

Scanned with CamScanner

* Heap Sort (InPlace)

A: | 100 | 20 | 30 | 10 | 15 | 7 | 16 |

[Sorted from behind (rev S)]

Algo

1. Build Heap (max elem)
2. Exchange root with last element, dec Heap Size
3. MaxHeapify, & repeat 2.

∴ MaxH → n times. on the root.
O(nlogn) → ∵ MH on the root can will be logn for atleast n/2 times
∴ log ∈ n/2.

Q. [10 8 7 6 | 5 1 2]
7 8 10.

* Bubble Sort

```
BS (int a[], n) ← adaptive BS
{
  for(i=0; i<n; i++)        → n-1 Passes
  {
    swapped = 0;
    for(j=0; j<n-i-1; j++).
    {
      if (a[j] > a[j+1])
      { swap;
        swapped = 1;
      }
    } if(swapped==0)
        break;
}
```

∴ $n-1+n-2+n-3+\cdots 1 = \frac{n(n-1)}{2} = O(n^2)$

→ Heaviest element comes to the right (outer bubble)

[10 | 5 20 | 40 20 50 | 40 | 60 | 15 70 | 70 15] Pass 1.

[10 20 40 50 15 | 60 70] Pass 2

→ will be done in (n-1) Passes.
→ if sorted Array; → we use swapped var
T = O(n)

→ [10 20 30 40 60 50] → in 2 Passes we can solve.

* Time Complexity Analysis

Scanned with CamScanner

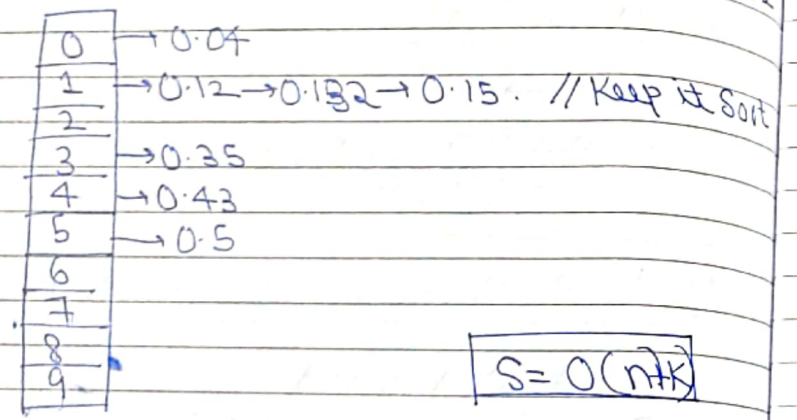
	findMax	delMax	insert	dec/inc key
maxHeap	O(1)	O(logn)	O(logn)	O(logn)

→ FindMin : O(n) [Present at leaf: n/2]
deleteAny : O(n) → O(n+n) → build Heap
Search : O(n)

* Bucket Sort

Sort a large set of floating nos in range 0.0 to 1.0 which are uniformly distributed across the range.

0.35 0.12, 0.43, 0.15, 0.04, 0.5, 0.132



S = O(n+K)

9 buckets → chaining [HT].

- Scan from top to bottom

T = O(n) [Best Case: Uniformly dist.]
T = O(n²) [Worst: All map into same bucket]

* Counting Sort

(Range must be known)

Restrictn: Key should be in a specific Range.
eg: Key ∈ (1,5)

A: [2 2 3 4 1 5 1 5] (n)

K	
1	2
2	X
3	1
4	1
5	2

2 → count occurrence of 2

for every no in array range, maintain a cell.

→ traverse the array
[1 1 2 2 3 4 5 5]

T = O(n+K)
S = O(K)

Disadv: • can't have any key beyond this range
• 1, 1000, 2, 3
→ 1000 size array will be req.
• range must be known

* Radix Sort (aid)

Q [804, 026, 005, 064, 052, 001]

• Base = 10
• make every no same digit.

Scanned with CamScanner

Apply stable sort

804	001	001	001
026	052	804	005
005	804	005	026
064	064	026	052
052	005	052	064
001	026	064	804
	(Unit)	(Ten)	(Hund)

Any

ALGO

// each key in A[1..n] is a d-digit no.
// Digits are numbered from right to left: 1 to d.

3 2 1
5 4

// for i = 1 to d:
  use Stable Sort Algo to sort A on digit i.

---

* Time Complexity (Radix Sort)

- When sorted using Unit digit, the 1 digit nos will be sorted.

- No. of digits in max no. = d.
- Use Counting Sort, if base b.
  Range → [0 to b-1]

$d * \underbrace{O(n+b)}_{\text{CountSort}}$ $\quad \boxed{d = \log_b l}$ (l = largest no.)

$\boxed{\log_b l\,(O(n+b))}$

If $l = O(n^k)$

$\Rightarrow \log_b n^k\,(O(n+b)) \Rightarrow O(n \log_b n)$

for $\log_b n \approx 1$, $n = b$
(no. of elements = base)
(it would beat comparison sorts).

$\boxed{T = \log_b l\,(O(n+b))}$
$\boxed{S = O(b)}$ ← Counting Sort.

---

* Selection Sort

- find the smallest elem in each pass, & place in beginning.
- Not adaptive.

$\underbrace{n-1 + n-2 + n-3 + \cdots 1}_{n-1 \text{ Passes}} = n(n-1)/2 = O(n^2)$

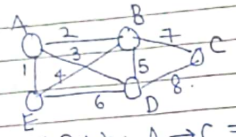
$\boxed{T = O(n^2)}$
$\boxed{S = O(1)}$

No. of Swaps (max) = O(n)

Scanned with CamScanner

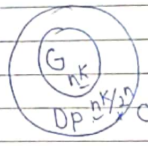
III] GREEDY ALGORITHMS

- For optimization Problem (Minimize/ maximize a Property)

eg 

Shortest Path: A → C = ?

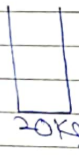
- $O(2^n)$: exhaustive Search. (P/A)
- Optimiz → Greedy (Want value all opt Prob).
  → DP (all Opt prob)
  - $O(nK)$ (perfect)
  - $O(2^n)$ (bad) → like Brute force



* KnapSack Problem → Sahani

n=3, m=20kg. // 3 obj, max capacity of bag = 20kg

	ob1	ob2	ob3
Profit	25	24	15
wt	18	15	10
P/W	1.4	1.6	1.5



I) Greedy obt Profits

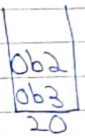
	w	p
~~ob~~ ob1	18	25
ob2	2	16/5

150 — 24
20 — x

$x = \frac{2 \times 24}{15} = 3.2$

Profit = 28.2 kg

II) Greedy obt wt



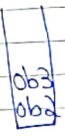
20 − 10 = 10.

15 — 24
10 — x

$x = \frac{10 \times 24}{15} = 16$ ₹

∴ P = 16 + 15 = 31 Knots

III) Greedy obt P/W ratio



20 − 15 = 5

10 — 15
5 — x

$x = \frac{5 \times 15}{10} = 7.5$ ₹

∴ P = 24 + 7.5 ₹

P = 31.5 ₹ (Max Profit earned).

Algo → [Fractional KS]

1. for 1 to n, compute Pi/Wi.
2. Sort object in dec order of Pi/Wi

Ob	1	2	3	4	5	6	7
Profit	10	5	15	7	6	18	3
Wt	2	3	5	7	1	4	1
	5	1.6	3	1	6	4.5	3

- Obj → (5, 1, 6, 3, 7, 2, 4) // dec order of P/W

2
7
3
Ob6
Ob1
Ob4

15

15 − 1 = 14.
14 − 2 = 12.
12 − 4 = 8
8 − 5 = 3
3 − 1 = 2

3 — 5
2 — x
x = 10/3.

∴ P = 6 + 10 + 18 + 15 + 3 + 10/3
⇒ 55.33

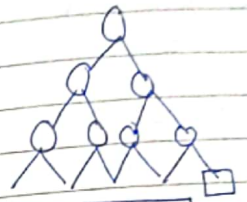
- Time Complexity: Algo → (Call)

O(n) + O(nlogn) + O(n) ⇒ O(nlogn)

T = O(nlogn)
S = O(1)

- Do Greedy KS using Max Heap:

1. Build max Heap (Pi/Wi) O(n)
2. delete max n times O(logn) × n.



logn = h. will be atleast n/2 times.

T = O(nlogn)

Q. n = 15, n = 5

Ob	1	2	3	4	5
P	2	28	25	18	9
W	1	4	5	3	3
P/W	2	7	5	6	3

Ob → (2, 4, 3, 5, 2)

5
3
4
2

15.

15 − 4 = 11
11 − 3 = 8.
8 − 5 = 3
3 − 3 = 0.

∴ 28 + 25 + 18 + 9 =

Scanned with CamScanner

→ Variable Size encoding

* Huffman Codes

- Compress a file (Zip compress)

100 char → made of a, b, c, d. How many bits?

f1

a-00, b-01, c=10, d=11 → fixed size encoding.

∴ 100 × 2 = 200 bits

→ If occurrences of char are not uniform HC.

	encode	occurrences
a : 50	0	
b : 40	10	
c : 5	110	
d : 5	111	
	codes.	

i) Highest freq → least bits. (Prefix Code)

a b c d a

0 10 110 111 0 → decode easily.

a b c d a (i) Prefix code).

[Code must not be prefix of any other]

→ ii) Prefix codes: Decode/encode easily.

a's : 50 × 1 = 50 bits

b's : 40 × 2 = 80

c's : 5 × 3 = 15

d's : 5 × 3 = 15

160 bits

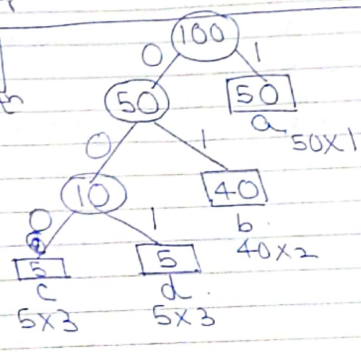
→ HC

∴ Non uniform encoding: reduce the no of bits based upon occurrence of the character

i) freq of char vary, → use HC.

a : 50
b : 40
c : 5
d : 5.
freq of char.

L:0 R:1 Convert



- a : 1
b : 01
c : 000
d : 001

Huffman prefix codes. [Same tree should be used for encode / decode]

∴ 50×1 + 40×2 + 5×3 + 5×3

⇒ 160 bits (req to save a file) } 40b saved.

uniform coding → 200 bits.

* 100 char → 160b

∴ No of bits/char = 1.6

or, [External Path length = No of bits to save file.

↓

Σ(freq × path length)]

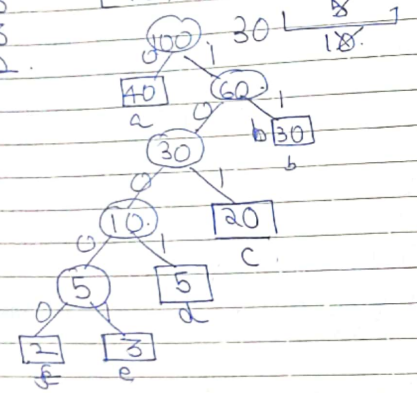
Scanned with CamScanner

Prob → build a tree, so as to obtain the external weighted Path length.

char	freq
a	40
b	30
c	20
d	5
e	3
f	2

No of char: 100

[40, 30, 20, 8, 5, 3, 2]



⇒ Pick least 2, add them, insert sum back.

∴ EPL = (40×1 + 30×2 + 20×3 + 5×4 + 3×5 + 2×5)

[bits req to code file] ⇒

EPL/100 = avg len/char

a = 0
b = 11
c = 101
d = 1001
e = 10001
f = 10000

Prefix Huffman codes (∵ way char is in leaf)

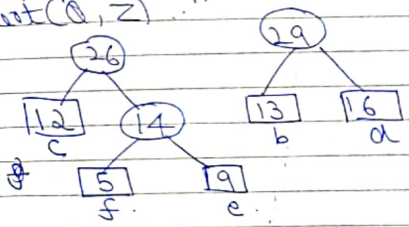
Algo (with q)

C: Set of all characters

a	45
b	13
c	12
d	16
e	9
f	5

Q: 45, 13, 12, 16, 14, 5 ← minHeap

1) Make a min heap with C
2) for i to n:
   allocate new node z
   z.left = extract min(Q)
   z.right = extract min(Q)
   z.freq = x.freq + y.freq
   Insert(Q, z)



Time Complexity

$O(n) + (n-1)[2 \cdot O(\log n) + O(\log n)]$

$T = O(n \log n)$

$S = O(n)$ → Space required to store tree.

Scanned with CamScanner

* Use Sorting Algo (Not Heap)

Sort → nlogn
extract min → O(n) } O(n) : Shift
insert → O(n)

n-1 times : $O(n^2)$

$T = O(n^2)$

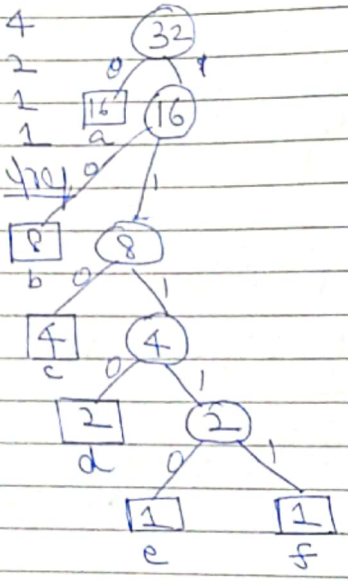
- if Problem : pulling out min & put it back → Use Heap
  if only pull out min → Sorting

* More examples on Huffman Codes

a	1/2	16
b	1/4	8
c	1/8	4
d	1/16	2
e	1/32	1
f	1/32	1

32 chars

Prob tree:



avg length = $\frac{16\times1 + 8\times2 + 4\times3 + 2\times4 + 1\times5 + 1\times5}{32}$

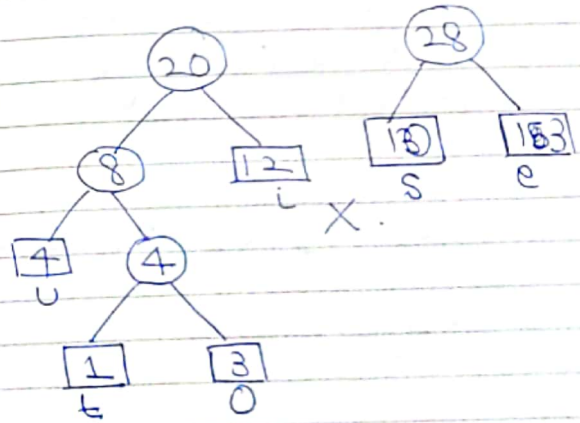
Q2

$\frac{1}{2}\times1 + \frac{1}{4}\times2 + \frac{1}{8}\times3 + \frac{1}{16}\times4 + \frac{1}{32}\times5 + \frac{1}{32}\times5$

⇒ 1.9375

Q

a	10
e	15
i	12
o	3
u	4
s	13
t	1



[we get a prefix]

* JOB SEQUENCING with Deadlines

Q

	J_1	J_2	J_3	J_4	
D:	2	1	1	2	← deadline
P:	6	8	5	10	

- You will get Profit, if you finish within deadline.

Scanned with CamScanner

<J4, J2, J1, J3>

· Maximise Profit:

0	1	2
J2		J4

⇒ 10+8 = ₹18 Profit

- Place Job as far as possible.

Q

	J1	J2	J3	J4	J5	J6
D	5	3	3	2	4	2
P	200	180	190	300	120	100

0	1	2	3	4	5
J2	J4	J3		J5	J1

← Gantt C.

→ <J4, J1, J3, J2, J5, J6> (Rev sorted acc to Profit)

⇒ ₹990 [Profit]

0	1	2	3	4	5
J2	J4	J3	J5	J1	

✓

Algo → Sort: O(nlogn)

· take each job, & search for vacant Pos/Slot: n × n = O(n²). (from max Deadline)

∴ T = O(n²)

S = O(d) → Max. Deadline X

→ no of Jobs.

Q

	J1	J2	J3	J4	J5
P	2	4	3	1	10
D	3	3	3	4	4

0	1	2	3	4
J1	J3	J2	J5	

Array Size: Max D = 4.

< J5, J2, J3, J1, J4>

= ₹19

Note: i) n << d → max deadline. n → no of Jobs, d → max Dead.
Space = O(n)

ii) n > d:
Space = O(d)

Q2

	J1	J2	J3	J4	J5	J6	J7	J8	J9
P:	15	20	30	18	18	10	23	16	25
D:	7	2	5	3	4	5	2	7	3

0	1	2	3	4	5	6	(7)
J2	J7	J9	J5	J3	J1	J8	

→ max Deadline = d.

∴ d < n

∴ S = O(d)

<J3, J9, J7, J2, J5, J4, J8, J1, J6>

X

⇒ Profit = ?

T = O(n²)

Scanned with CamScanner

* Optimal Merge Pattern

- Node ka jisme width record ko store karne ke liye ho.

- Jab 10 records ko ek jagah se dusri jagah move karna ho.

- EPL = no of record movement
= 10 x 2 + 20 x 2 + 30 x 1

⇒ add minimal Node (60+30)

↳ like minimum weighted external Path length [Huffman coding]

1. Create minHeap : O(n)
2. Doko a min, main it & put it back : O(logn)
n-1 times : O(nlogn)

T = O(nlogn)

* Maintain Heap

- Node ka jisme width record ko store karne ke liye ho.

- Jab 10 records ko ek jagah se dusri jagah move karna ho.

- EPL = no of record movement
= 10 x 2 + 20 x 3 + 30 x 4 + 40 x 5 + 50 x 6
= 10 + 40 + 90 + 160 + 250 + 300
= 850

- 850 record mov minimum
(Largest record plus will be moved last, means to the root)
(Smallest item or last item root)

* Intro to Spanning Trees / Kirchoff Theorem

- Graphs → (V, E)

→ Simple (no self loop / multi loop)
→ Multigraph

→ Have, simple Graphs

* Null Graphs : no edges

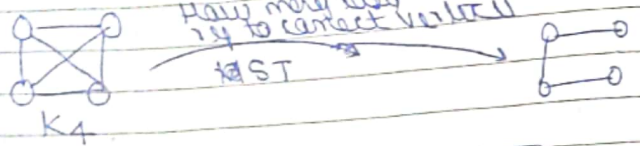
* Max edges in Simple Graph = $\frac{n(n-1)}{2}$

min edges = 0

Scanned with CamScanner

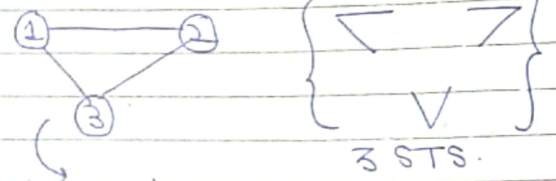
$E = O(V^2)$
$V = O(\log E)$
→ Worst Case

* Weighted Graph & Unweighted Graph
[Cost/distance]



Min edges req to connect n Vertex → (n−1)

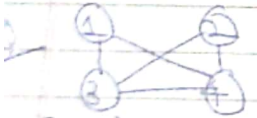
- Spanning tree: SubGraph



labelled Graph [Nodes have names] : 3 STs.

No of STs, for $K_n \longrightarrow n^{n-2}$ STs
[Complete Graph]

* No of STs for a Graph (which is not complete) ∴ STs ↓



[Undirected Graph]

* Kirchoff's Theorem (Simple Graph: Diagonal 0)

- Construct Adjacency Matrix

	1	2	3	4
1	0	0	1	1
2	0	0	1	1
3	1	1	0	1
4	1	1	1	0

(i) Diagonal 0s → degree of nodes
(ii) Non diagonal 1s → " −1 "
(iii) Non diagonal 0s → 0s "

	1	2	3	4
1	2	0	−1	−1
2	0	2	−1	−1
3	−1	−1	3	−1
4	−1	−1	−1	3

No of STs = Cofactor of any element
$= (-1)^{i+j} M_{ij}$

$\text{Cof}(11) = (-1)^2 \cdot [2\cdot(9-1) - (-1)[-3-1) + -1(1+3)$

$= [16 - 4 - 4) = \underline{8}$

∴ No of STs = 8

Scanned with CamScanner

Q K_3 →

	1	2	3
1	0	1	1
2	1	0	1
3	1	1	0

	1	2	3
1	2	-1	-1
2	-1	2	-1
3	-1	-1	2

⇒ $STs = 4 - 1 = 3$

No of STs = 3

Q

	1	2	3
1	0	1	0
2	1	0	1
3	0	1	0

	1	2	3
1	1	-1	0
2	-1	2	-1
3	0	-1	1

$2 - 1 = 1$. No of STs = 1

Q

→ 16 (KT)

* MST and Example on Prim's Algorithm

MST → edge to connect all vertex

⇒ Weighted Graph

Undirected graph with min weight.

Q

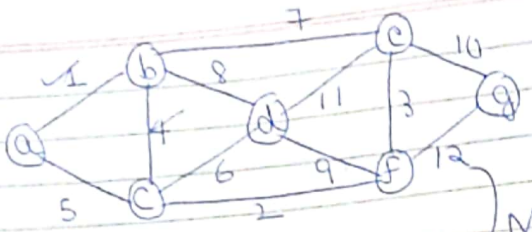
⇒ 60 (MST)

Note: find out min ST.

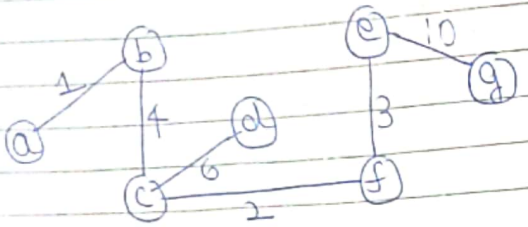
Blue lines: 1 out of n^{n-2} : exp

* Prim's Algorithm → find out MST

Q



MST.

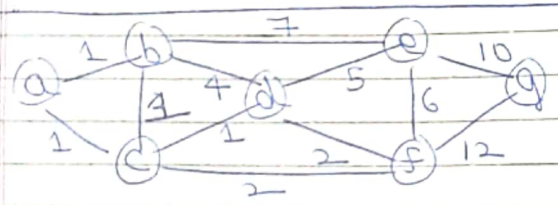


→ Prim → a tree grows one edge at a time

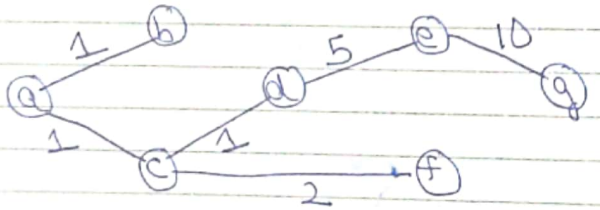
→ (at any time, we get a tree)

→ We never get a cycle here ∵ we add a new node outside the tree (unvisited nbrs)

Q



→ If edges will not distinct, we might get more than 1 MST (like this Q).



→ sort have all the edges of least wt in MST.

b - e d
c - d ~~8~~.
d - e ~~8~~.
f - e g.
MST: 20

if ab chosen:
- check unvisited nbrs of a, b, & pick the one with min distance.
[c, d, e]

a: c
b: c, d, e.

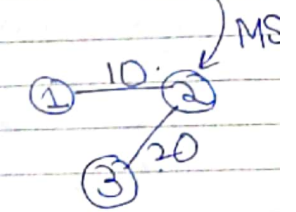
∴ (n-1) edges must be present in ST.

Gate-10

	0	1	2	3	4
0	0	1	8	1	4
1	1	0	12	4	9
2	8	12	0	7	3
3	1	4	7	0	2
4	4	9	3	2	0

Q

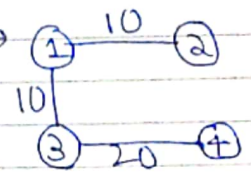
	1	2	3
1	0	40	30
2	40	0	20
3	30	20	0



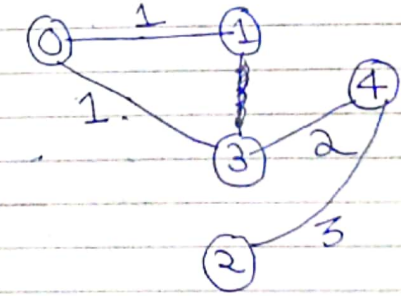
Q

	1	2	3	4
1	0	10	10	50
2	10	0	40	30
3	10	40	0	20
4	50	30	20	0

MST



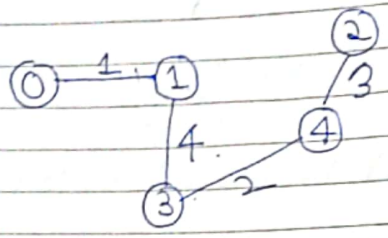
40



Scanned with CamScanner

	0	1	2	3	4
0	0	1	8	1	4
1	1	0	12	4	9
2	8	12	0	7	3
3	1	4	7	0	2
4	4	9	3	2	0

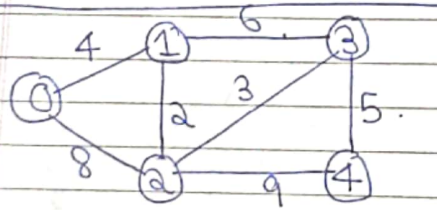
MST st. 0 shud be a leaf



cost = 10

* Prims Implementation without MinHeap

- (n×n) adjacency matrix
- MST → t [all edges, undirected]
- $T = O(n^2) = O(V^2)$



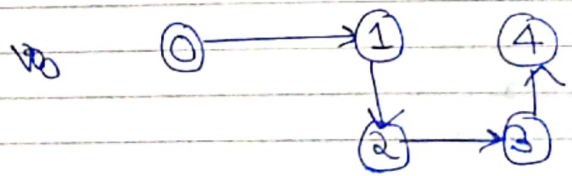
V_0 ✓	V_1	V_2 ✓	V_3 ✓	V_4
0	∞	∞	∞	∞
	4	8	6	9
		2	3	5

- boolean visited[V], weights[V], parent[V] → 3 DS used.
- → Run loop V times → O(V)
- {
  - (V) · Pick vertex with minWeight (min) from unVisited
  - · Mark it as visited.
  - · explore all its unvisited nbrs:
  - (V) → decide whether to update parent[] & weight[]
    - if(weights[i] > edges[min][i]) (edges → costMatrix)
- }

V_0 ✓	V_1 ✓	V_2 ✓	V_3	V_4
0	∞	∞	∞	∞ ← Initially
0	4	8	∞	∞ ← Pick 0 as minWt. & update nbrs & parent (Pick V_1)
0	4	2	6	∞ ← V_2 is chosen from UnV.
0	4	2	3	9 ← V_3
0	4	2	3	5 ← V_4

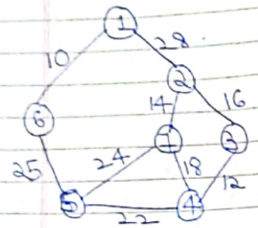
→ The parent[] contains the MST.

	0	1	2	3	4
Parent	-1	0	1	2	3



Scanned with CamScanner

* Prim's Algorithm (Min Heap)



- Heap is used to choose the next min vertex (to be added into Tree)

→ G: Graph, Cost: Matrix, r: root. [random node]

for every Vertex → Key, parent. [cost].

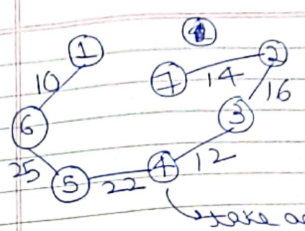
	1	2	3	4	5	6	7
Key	~~∞~~ 0	~~∞~~ 28	~~∞~~ 12	~~∞~~ 22	~~∞~~ 25	~~∞~~ 10	~~∞~~ ~~24~~ 18
Parent	N	~~N~~ 1	~~N~~ 4	~~N~~ 5	~~N~~ 6	~~N~~ 1	~~N~~ ~~5~~ 4

0 → minHeap.

→ (1) is extracted.

5 → Build Heap

11 → decrease Key



take adj. nbr of 4 naw, & w if key dec.

Algo → Code

→ Build Heap: O(V)
while loop: V time
{
- extract min: O(log V)
- Decrease Key: O(V log V) → Aggregate Analysis → O(E log V) ∵ decrease key is actually called E times.
- Build Heap: O(V)
}

→ T = O(V log V + E log V + V)

T ⇒ O(E log V) → MinHeap

minHeap for Prim's.

- Fibonacci Heap → decrease Key is constant time.

∴ T = O(V log V + E) → Fibonacci Heap

T = O(V²) → Cost Matrix.

Scanned with CamScanner

Scanned with CamScanner

→ comparison of 2 imp

Q $O(E \log V) \longleftrightarrow O(V^2)$

In case, Dense Graph (lot of edges).

$E = O(V^2)$.

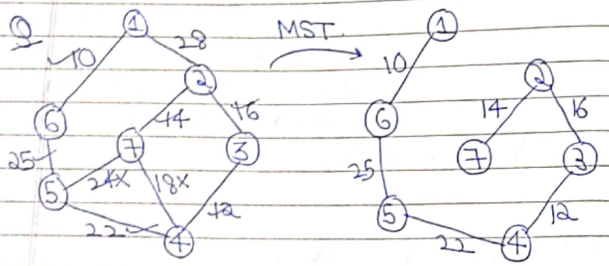
$\therefore O(V^2 \log V) > O(V^2)$ — Cost Matrix is better

In case, Sparse Graph (— Heap is better.

$E = O(V)$.

$O(V \log V) < O(V^2)$

* Intro to Kruskal's Algorithm & Examples on it.

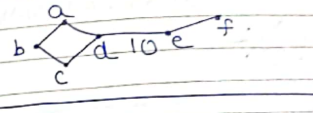
Q 

- Kruskal → we may get forest.
- See, that it doesn't form a cycle.

Prims: no need to check for cycle
↳ add a node (unvisited).

Q 2000:

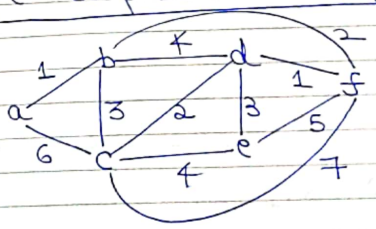
- MST can only be found out from Graphs which can be connected.
- if distinct edge weights → MST unique.
- a) every MST must contain mine.
- b) if mine in MST, then its removal must disconnect G.



→ if repeated edge wts

Q edge e with wt. w.

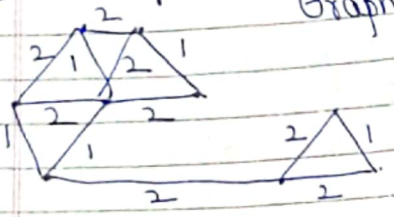
a) There is a MST containing e'. ✓
b) if e not present in MST, all edges in the cycle have same weight. ✓
(c) ✗ ✓
(d) e' not present in every MST.

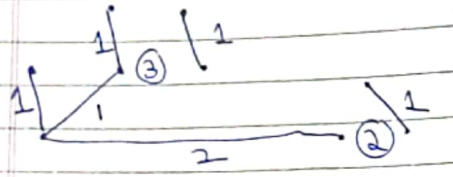
Q 

[Which among these is OK with Kruskal's]

a b d f
c e

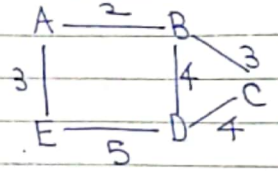
Q No. of distinct MST for the weighted Graph below:

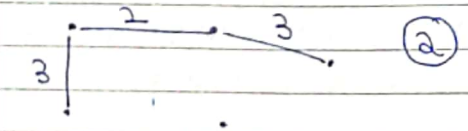




⇒ 3 × 2 = 6 Ways.

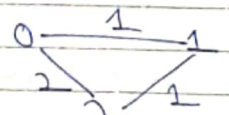
Q





∴ 2 MSTs are possible.

Q



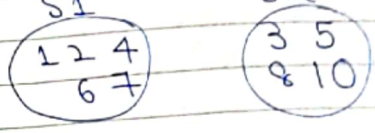
* Amortized Analysis

* Data Structure for Disjoint Sets + Kruskal

→ next.

* Data Structures for Disjoint Sets

- Grouping certain elem → set



DS which groups elem into sets, per opern &

[Disjoint Sets (no return b/w sets)

Opern

1) 1: which set ? → S1
2) ∪ union → S3

- A disjoint set DS, maintains a collection S of pairwise disjoint sets.

dynamic
[ins/del at run time]

* Opern on Disjoint Sets

1: Create-Set(x) : create new set with elt

2

[Choose elem in each set as rep]

S1 : 1 , S2 : 3 , S3 : 6

Set(2) → 1 [rep] → representative

(2, 5)
rep ↓ rep ↓
1 3 ∴ diff sets.

(choose any node as representative).

Scanned with CamScanner

2. Union.

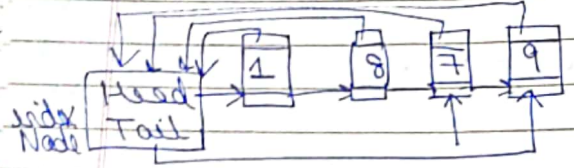
$S_1 \cup S_2$ [old sets destroyed]

S_3 add to Disj sets.

3. Find_set(x) : find set of x.

* Linked list rep of Disjoint Sets

$S_1 = \{1, 7, 8, 9\}$ → no order.



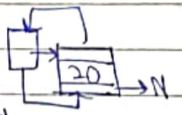
Here, representative = 1 (first obj)

→ each set is LL, & each elem = object.

[Set Member, next Pointer, Index P].

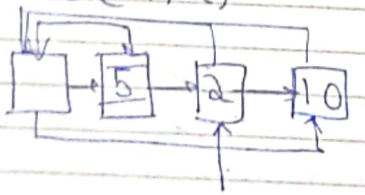
* Oprn

1. Create_set(20) → O(1)



2. Find_set(7) [pointer] → O(1)

3. Union(7, 2) → 2 Pointers.

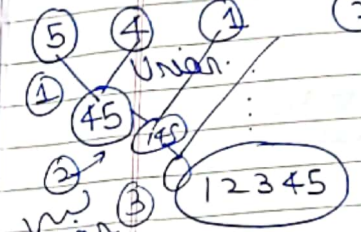


O(n)

- change tail to head & traverse with, to change pointers to new head.

∴ O(n)

* Amortized Analysis



→ 5 Disj Sets.

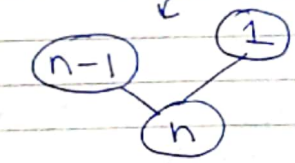
∴ n create oprn, n-1 union oprn.

∴ n+n-1 = 2n-1 oprations.

⇒ Create Set = O(1) × n = θ(n)

⇒ Union = 1 + 2 + 3 + 4 ⋯ n-1. [Worst case]

= θ(n²)



∴ O(n²)

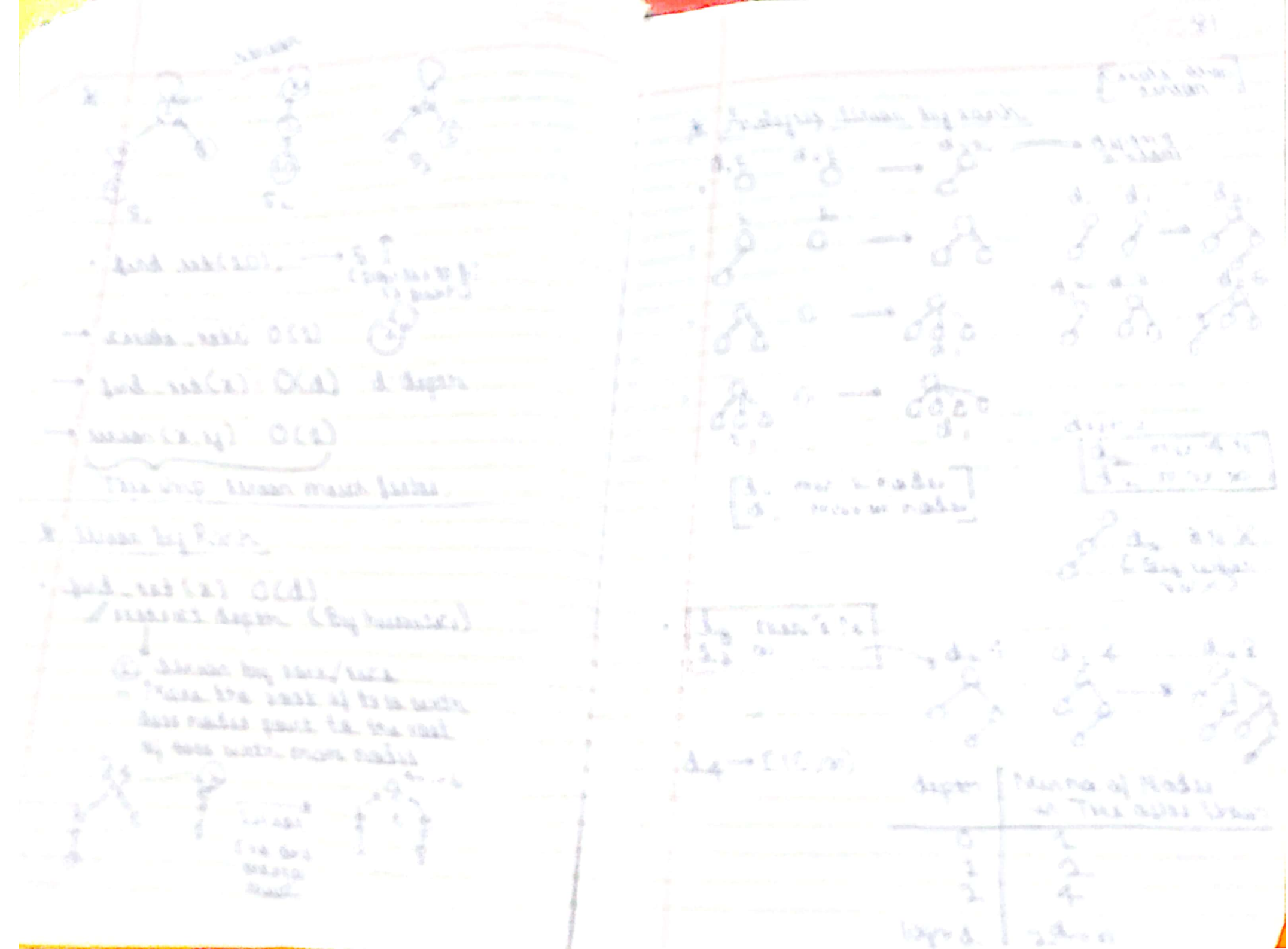
∴ for n oprns = $\frac{O(n^2)}{n}$ ⇒ O(n).

$\boxed{T = O(n)}$

* Disjoint Set Forest (Tree Implem)

- Pointers from child to parent.
- each tree: set, root → representative.

Scanned with CamScanner



Scanned with CamScanner

→ We need to have n nodes)

- Max depth possible with n nodes = $\log n$
- For depth d, min nodes req = 2^d

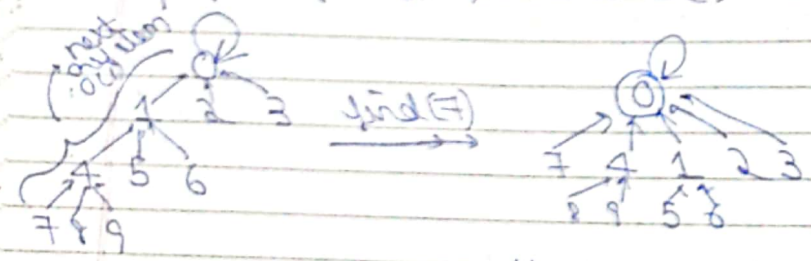
n nodes ⟶ max depth = $\log n$

∴ **Find = T(O log n)**

* Path Compression

⇒ Restrict depth

— whenever we perform find(x), make every node visited in the path point to the root. Next time, when we perform find(x), it takes O(1)



Next 7: O(1).

- Combine Union By Rank & Path compress

* Time Complexity Using Heuristics

1 Union: O(1)
2 Create: O(1)

3 Find ⟶ O() (Amortized Analysis)

- O(m log n) Union By Rank (m ops)
- O(n + f(2 + log n)) Path Compression
  (2+1/n)
  ↳ Create Set

∴ With Union By Rank, PC

- $O(u + f(\alpha(f+u, u)))$ for a sequence of u union & f Find- Set operations
  α ⟶ slow growing function
  [Inverse Ackermann function]

T ≈ O(u+f)
(u+f)

∴ **T = O(1)** ⟶ find(x)

* Connected components using Disjoint

1) Use BFS/DFS to see if G is connected or not / to find no of connected components [Static Graphs].

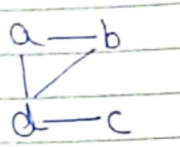
2) For dynamic Graphs, BFS/DFS X.
Use Disjoint Set DS ⟶ **T = O(E), S = O(V)**

- for every vertex create Set

Call ⟶ Algo

Scanned with CamScanner

* Example



Init: {a} {b} {c} {d} {e} {f} {g}

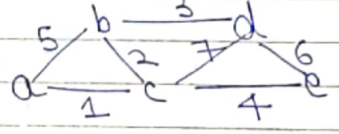
edge: (a,b) : {a,b} (Union)

→ (a,d)	{a,b,d}, {c}, {e}, {f}, {g}
→ (e,f)	{abd}, {ef}, {c}, {g}
→ (b,d)	Same.
→ a,c.	{abdc} {ef} {g}

3 Connected Components.

* Kruskal Algo using Disjoint Sets

→ Find MST.




```
{
  A ← ∅                                   → Union By R/ Path Comp.
  for each vertex, makeSet(v)  : O(V)
  E' → Sort all edges in inc order        O(ElogE)
  for each edge (u,v) ∈ E'
  { if( Find_set(u) ≠ find_set(v) )       → if no cycle [diff sets]
      A ← A ∪ {(u,v)}   // add into MST
      Union(u,v)
  }
}
```

→ E·O(1) = O(E)

∴ O(E + V + E log E)

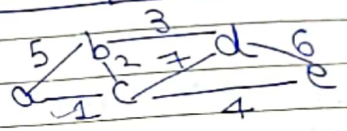
⇒ O(E log E) E = O(V²)

∴ T = O(E log V²)

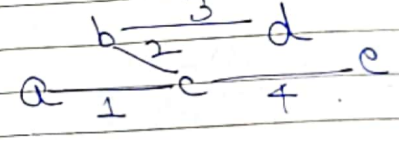
T = O(E log V) / T = O(E log E)

S = O(V)

* Example of Kruskal



• {a} {b} {c} {d} {e}



• ac : {ac} {b} {d} {e}.
• bc : {abc} {d} {e}.
• bd : {abcd} {e}
• ce. : {abcde}
• ab : same set X [Create cycle]
• de : same set X.

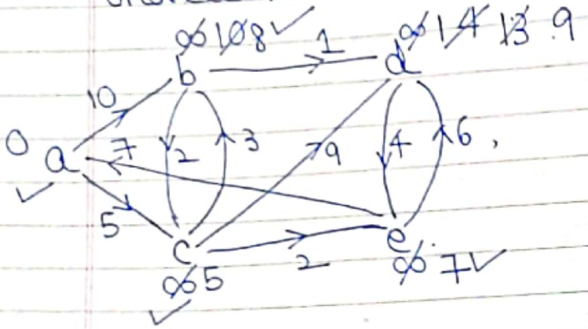
} count of edges to track (n−1)

Scanned with CamScanner

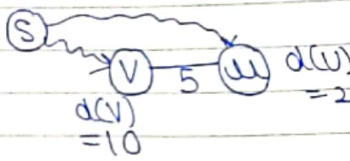
path problem.

* Dijkstra Algorithm [Weighted & Directed]

- Shortest Path (S → D).



* Relaxing an edge : O(1) → depends on DS we use.



[In Algo, SP from S to u : 20]

∴ d(v) + C[u,v] < d(v)
d(v) = d(v) + C[u,v] (Updated to 15)
∴ Edge relaxed.

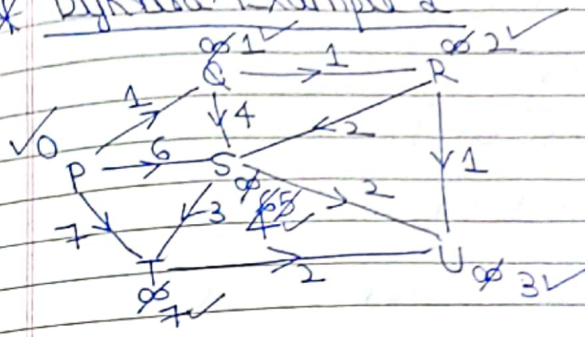
→ Dijkstra doesn't work with -ve edges [some cases work]

```
Dijkstra()
{
  • Pick a vertex (with min wt)
  • Relax all edges (nbrs) from it, not chosen nodes
}
```

[Relax an edge only if it's not considered before]

Ans → a b c d e
0 8 5 9 7 : Shortest Path from Src.

* Dijkstra: Example 2



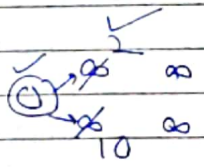
[Start. Relax only outgoing edges]

P Q R S U T.
0 1 2 4 3

[P Q R U S T] : Order of relaxation (SP)

* Dijkstra Algo → Heap is built.

```
Dijkstra(G, w, s)        → Set of vertices whose SP decided
{
O(V): Initialize values (0, ∞ ∞ ∞ ...)
      • S = ϕ
O(V): Q = G.V   (Vertices in Heap)
      • while Q ≠ ϕ : // Heap empty
          u = extract-min(Q)
          S = S ∪ {u}
          for each vertex v ∈ Adj(u)
              relax(u, v, w)
                  → decrease Key:
}
```



- extract min : O(log V) → V times : O(V log V)
  Decrease Key : O(log V) → E times : O(E log V)
  [in worst case, each edge relaxed]

Scanned with CamScanner

$T = O(V + V + V\log V + E\log V)$

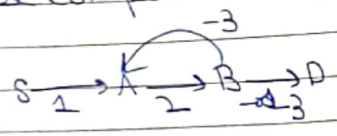
$\therefore \boxed{T = O(E\log V)}$

[If min-Heap used]

* Negative weight Edge & Dijkstra

(ii) If Graph is disconnected, Shortest Path not Possible.

(iii) If -ve edge cycles, then SP cant be computed.



$SP = -\infty$

but, -ve edge with : ⊗ SP possible (not cycle)

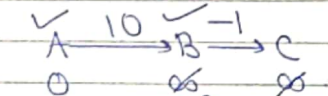
Algo (Dijkstra cont).

-ve Cycle — -ve Cycle X.

→ if Algo cant identify -ve edge cycle, it will give wrong ans : Dijkstra

∴ Avoid -ve edge in Dijkstra.

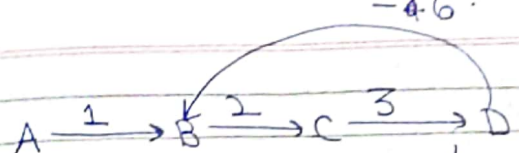
Q



	A	B	C
~~10~~ 0 A	0	∞	∞
10 B		10	∞
9 C			9

Dijkstra is correct : Hen.

-46.



	A	B	C	D
A 0	0	∞	∞	∞
B 1		1	∞	∞
C 3			3	∞
D 6				6

Wrong ans by Dijkstra (Due to -ve edge cycle)

∵ Vertex deleted from Heap, when SP computed.

∴ SP doesn't exist.

* Dynamic Programming → similar to DnC

dynamically table decide

* dnC : divide & merge soln.

⇒ if overlapping SP, dnC computes again & again.

↳ DP stores it.

* Fibonacci

$f(n) = f(n-1) + f(n-2)$

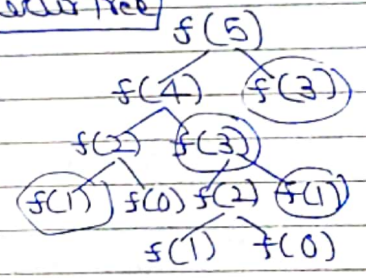
$\Rightarrow 1, n = 1$

$= 0, n = 0.$

$\therefore T(n) = T(n-1) + T(n-2)$

$\boxed{T = O(2^n)}$

Recursion Tree



f(1), f(3) repeated (computed again & again)

Scanned with CamScanner

∴ Save it in a table

2	3	4	5
2	3	5	

T=O(n) (Memoization/Program)

- if table is empty, then only call it, else retrive from table.
  ∴ dynamically decide

* In merge Sort → DP X
  ∵ [no overlapping SPs]

* Matrix Chain Multiplication

* Matrix Multiplication

A: (3x2) B

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}_{3\times2} \quad \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix}_{2\times3} = \begin{bmatrix} \text{Scalar} \\ 2 \times \text{mult} \end{bmatrix}_{3\times3}$$

A B C

[9 elements]

∴ 18 mult.

∴ Total mul = 3x3x2
= 18 mul.

Total mul = p x r x q

(P x R) · (R x Q)

Size: P x Q

* Chain Multiplication Introduction

$A_{2\times1}$ $B_{1\times2}$ $C_{2\times4}$

Ans will be same

1. (AB)·C
2. A·(BC)

$^{2n}C_n$ / $n+1$ → no of parenthesis ways.

1x1 1x4 → 8 + 8 = 16 mul

2x2 2x4 → 4 + 16 = 20 mul (2x4)

∴ Less cost.

→ The way we Parenthesize, is going to impact #mult.

* Examples

ABCD

A(BCD) (AB)·(CD) (ABC)D ⇒ 5 Ways

A(B(CD)) A((BC)(D)) A·(BC)D ((AB)C)D

→ which mult is better: depends upon mul

$A_{1\times2}$, $B_{2\times1}$, $C_{1\times4}$, $D_{4\times1}$

1) A(B(CD)) = 4 + 2 + 4 ⇒ 10 mul

1x2 2x1 1x1

2x1

1x1

Scanned with CamScanner

2) $A_{1\times2}\left(\underbrace{(BC)}_{2\times4}\underbrace{(D)}_{4\times1}\right)$ $\underbrace{\qquad\qquad}_{1\times1}$ $= 8+8+2$ $\Rightarrow$ 18 mul

3) $\underbrace{(AB)}_{1\times2}\underbrace{(CD)}_{1\times1}$ $= 2+4+2$ $= 8$ mul

4) $((AB)C)D$ $\underbrace{\quad}_{1\times4}$ $= 2+4+4$ $= 10$ mul

5) $\underbrace{(A\underbrace{(BC)}_{2\times4})}_{1\times4}\underbrace{D}_{4\times1}$ $= 8+8+4$ $= 20$ mul

∴ But $\longrightarrow$ (AB)(CD)

* if 5 Matrices, no of ways to Parantheses

ABCDE $\longrightarrow$ (ABCD)E, 5

A(BCDE), (AB)(CDE), (ABC)(DE)

5, 1, 2, 2

. 14 Ways

For n matrices: $\frac{(2n)!}{(n+1)!\,n!}$

** if n=4 $\longrightarrow$ $\frac{2n C_n}{n+1}$

subst n=3 [n−1]

* $\frac{2n C_n}{n+1}$ $\longrightarrow$ no of BST / no of BTS possible with n unlabelled nodes

* for DP:

① Optimal Substructure: if Given big Problem, there must be SPs, such that: Soln to small Problem put together, solves the big Problem.

② Overlapping SubProblems (if no Overlapping $\longrightarrow$ DnC)

* Optimal Substructure & Recursive Eqns

requirement for DP.

Optimal SS $\longrightarrow$ Split big Problem into Smaller optimization Problems.

. $A_1 A_2 A_3 \cdots A_n$ (dim of A_n: $P_{n-1}\times P_n$)
$P_0\times P_1$ $P_1\times P_2$ $P_2\times P_3$

$(A_i \longrightarrow P_{i-1}\times P_i)$ dim

. $A_i A_{i+1} A_{i+2} \cdots A_j \longrightarrow (A_i A_{i+1} \cdots A_K)(A_{K+1} \cdots A_j) +$ $P_{i-1}\times P_K$, $P_K\times P_j$

$i \le K \le j$ $P_{i-1} P_K P_j$

$(A_{i\cdots K})(A_{K+1\cdots j})$

Substructure

K = Split

$A_i\ A_{i+1} \mid A_{i+2} \cdots A_j$ (split at K)

* Recursive equation

$m[i,j]$ → min no. of Mul required to mul $(A_iA_{i+1}\cdots A_j)$

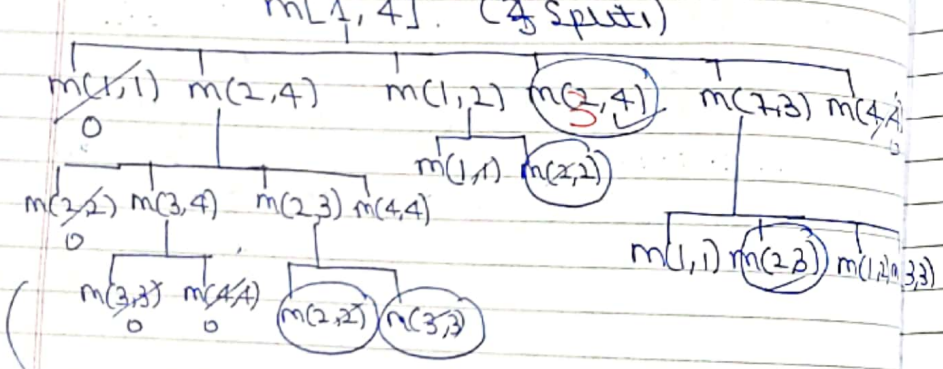
** $$m[i,j] = \begin{cases} 0 & ; i=j \\ \min\left\{ \underbrace{m[i,K]}_{P_{i-1}\times P_K} + \underbrace{m[K+1,j]}_{P_K \times P_j} + P_{i-1}P_K P_j \right\} & \\ i \le K \le j-1 & \end{cases}$$

Recursive eqn. — Top Down

$K \in [i, j-1]$

* Recursion Tree

$m[1,4]$: 4M $A_1A_2A_3A_4$.



→ DP traversed to worst.

∴ depth = $O(n)$

∴ $T = O(2^n)$ → No of nodes in a complete BT.

(no of function calls)

But, have lots of overlapping SPS.

∴ max unique fcs ⇒ $O(n^2)$ [4·4]

1) Bottom up (Table) — No Recursion } DP
2) Memoization — Recursion.

* DP - Bottom up Implementation

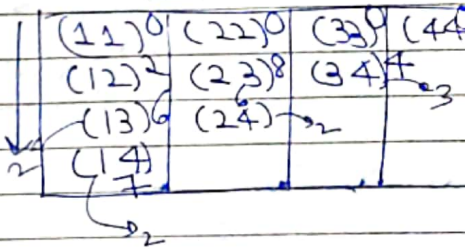
Size					
1	(1,1)	(2,2)	(3,3)	(4,4)	: 4 } n^2 unique function calls.
2	(1,2)	(2,3)	(3,4)		: 3
3	(1,3)	(2,4)			: 2
4	(1,4)				: 1

∴ for $A(1\cdots n) = n + n-1 + n-2 + \cdots 1$

$= \frac{n(n+1)}{2}$ Subprob.

∴ **Unique SPS = $O(n^2)$**

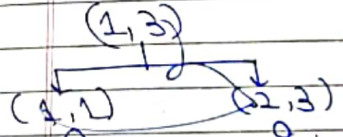
Table



Q $A_1\ A_2\ A_3\ A_4$

$1\times2\ \ 2\times1\ \ 1\times4\ \ 4\times1$

$P_0\ P_1\ P_1\ P_2\ P_2\ P_3\ P_3\ P_4$



$$m(1,3) = \min \begin{cases} \underset{0}{(1,1)} + \underset{8}{(2,3)} + P_0P_1P_3 = 16 \\ (1,2) + (3,3) + P_0P_2P_3 = 6 \\ \ \ 2 \quad\ 0 \quad\ \ 8\ 4 \end{cases}$$

Scanned with CamScanner

$$(2,4) = \min \begin{cases} (2,2) + (3,4) + P_1P_2P_4 = 16 \\ (2,3) + (4,4) + P_1P_3P_4 = 16 \end{cases}$$

(0, 4, 2; 8, 0, 8)

3 Split $(1,4) = \min_m \begin{cases} (1,1) + (2,4) + P_0P_1P_4 = 8 \\ (1,2) + (3,4) + P_0P_2P_4 = 7 \\ (1,3) + (4,4) + P_0P_3P_4 = 8 \end{cases}$

(1×2, 2×1, 1×4, 4×1)

Split at 2

if $m(1,n) = n$ Splits.
$\Rightarrow O(n)$ // to compute 1 entry

$\therefore$ $T = O(n^2 \times n) = O(n^3)$

- $(1,4) = ((1,2)(3,4))$
  See By Split: [From left cell]

* Iterative Tabular Method.

$S = O(n^2)$

* Algo → Code

n = 4

A_1	A_2	A_3	A_4
1×2	2×1	1×4	4×1

dimensions.

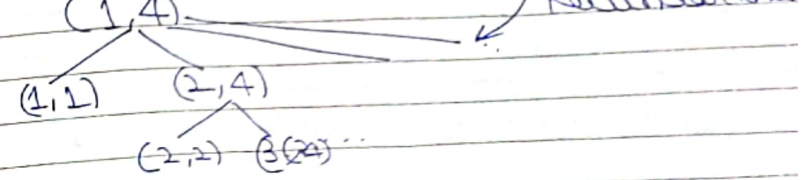
P → [1 2 2 1 1 4 4 1]

P_0 P_1 P_2 P_3 P_4

- We only need 1/2 of Matrix.
  S → Splits.

$T = O(n^2 \times n) = O(n^3)$.

* Top Down DP: Memoization Problem



- b4 calling funct, check if val is in table, if not call, else return.
- Only n^2 FCs are made.

$T = O(n^3)$, $S = O(n) + O(n^2) =$

depth of Recursion Tree

$S = O(n^2)$

Same like bottom up Approach.
(Actual space: bit more)

* Longest Common Subsequence

RAVINDRA
Subseq: RAVI, NRA, AI, AR ... (can be non-cont)

For n length String: 2^n SS

Scanned with CamScanner

App^n: DNA → Strand of molecules (ACGT)

D1: AGCCTCAGT··· ← DNA 1
D2: GCCT ← DNA 2.

(4 DNA SS of other, very similar)
→ Find the LCS (for similarity checking).

Q RAVINDRA — n
AJAY — m
LCS = {AA} = 2
{}.

① Brute Force: Str 1: m = 2^m SS & check with S2.
Str 2: n.

T = O(2^m n)
[Exponential]

∴ O(2^m × n)

→ Apply DP → Optimal SS
→ Overlapping SPS.

* Optimal Substructure & Recursive eq^ns

X $x_1 x_2 x_3 \cdots x_n$
Y $y_1 y_2 y_3 \cdots y_m$

→ We get OS.
$x_1, x_2 \cdots x_i$
$y_1, y_2 \cdots y_j$

** $C[i,j] = \begin{cases} 0, & i=0 \text{ || } j=0 \\ 1 + C[i-1, j-1]; & i,j>0 \ \& \ x_i = y_i \\ \text{max}\begin{cases} C[i-1,j] \\ C[i,j-1] \end{cases}; & i,j>0 \ \& \ x_i \neq y_j \end{cases}$

(Recursive eq^n)

* Recursion Tree & Unique S.P

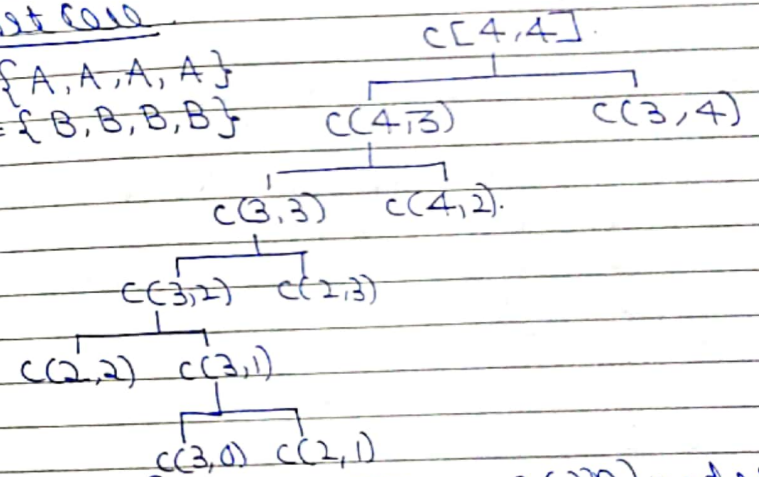
1) X = {A, A, A, A} → Best Case
Y = {A, A, A, A}

C(4,4) ⇒ 4
1 + C(3,3)
1 + C(2,2)
1 + C(1,1)
1 + C(0,0) 0

∴ T = O(n)

2) Worst Case.

X = {A, A, A, A}
Y = {B, B, B, B}



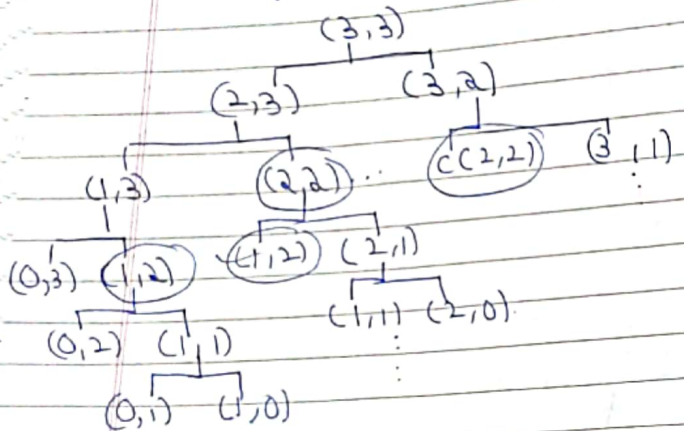
∴ depth of tree = 2n, full BT ∴ O(2^{2n}) nodes.
depth ⇒ O(n+m)

T = O(2^{n+m}) (exp time).
→ like Brute force.
[Wastes more increased Space].

→ lots of Overlapping SPS

Scanned with CamScanner

∴ No of Unique SPs = O(n×m)



∴ Non Overlapping SP.

(3,3)

(2,3) (1,3) ...

⇒ O(n×m) Unique function calls

→ construct a table, & memoize.

or, Bottom up DP

	0	1	2	3
0	O_{00}	O_{01}	O_{02}	O_{03}
1	O_{10}	11	12	13
2	O_{20}	21	22	23
3	O_{30}	31	32	33

O((m+1)(n+1)).

T=O(nm), S=O(nm)

Q2. Top Down Memoization method.

* eg 1

X: AABB → LCS=2 BB

Y: CBDB

or

X: AAB → AA

Y: ACA.

		A	A	C	A
		0	1	2	3
Y→					
X	0	0	0	0	0
A	1	0	1	1	1
A	2	0	1	1	2
B	3	0	1	1	2

AA → LCS

* example 2

X: {A B C B D A B}

Y: {B D C A B A}

		0	1	2	3	4	5	6
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

BCBA → LCS

LCS = 4

BCBA / BDAB : depend upon path we take from Bottom.

3 SS

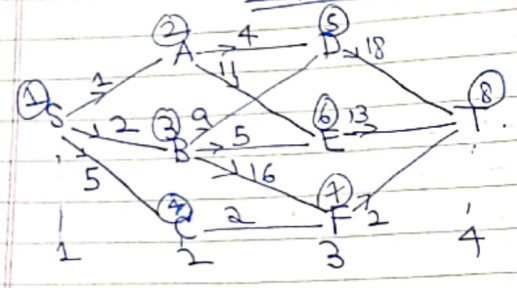
Scanned with CamScanner

* Bottom up DP:

$T = O(mn)$

* Multi Stage Graph

At every Stage: some nodes like Bipartite.



Chars

1) Within 1 stage no edge.
2) edge only S_i to S_{i+1} only.

Q) From S → T: Find Shortest Path?

- Use Dijkstra Algo:
  - → SP between Source to all Nodes (Lot of effort).

$T = O(E \log V)$

* Greedy will take 1,
  - → when with optimiz^n Problem,
    No guar. → correct.
    Adv: faster.

∴ Greedy fails ✗.

S — A — D — T
1 4 18 = 23

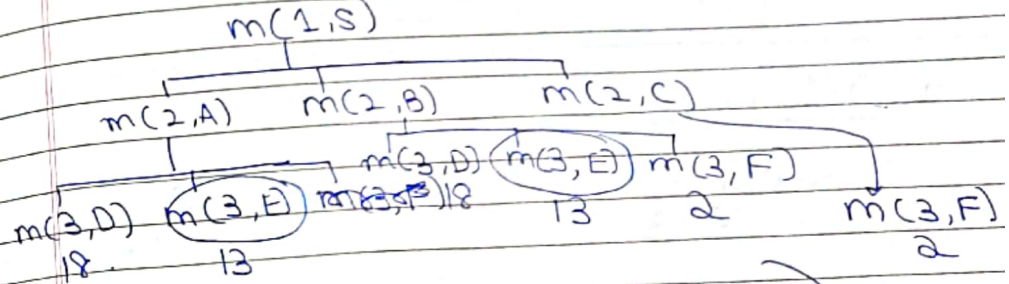
But, 9 is best.

DP → SubProblems

$m(N-i, i) = i \to T$

$$M(1,S) = \min \begin{cases} S-A + m(2,A) \\ S-B + m(2,B) \\ S-C + m(2,C) \end{cases}$$

Stage 1 node's



depth = O(n) // no of Stages.

$T = O(2^n)$ → exponential.

Lots of overlapping SPs.

- No of Unique SPs = No of Vertices = n.

$S = O(n)$ → Array [1D].

* Bottom up DP

- No the Vertices.

$$T[i] = \min_{\text{for all } v+1 \text{ to } n} \{ \text{cost}(i,v) + T[v] \}$$

T	1	2	3	4	5	6	7	8
	9	22	18	4	18	13	2	0

Ans

$$T[6] = \begin{cases} \text{cost}(6,7) + T(7) \\ \text{cost}(6,8) + T(8) \end{cases}$$

∞ 2 ← 8a from here

13 2

Scanned with CamScanner

→ Unique SP = O(n).

∴ T = 1+2+3+···· n-1

⇒ n(n-1)/2

T ⇒ O(n²) ; S = O(n)

⇒ O(V²)

= O(E)

∴ T = O(E) S = O(V)

* 0/1 KnapSack

KS

fract KS — eg: rice — Greedy

0/1 KS — eg: Pants — DP.

Capacity = 6.

Ob	1	2	3
Wt	1	2	4
Prof	10	12	28

Greedy

P/W : 10, 6, 7.

→ Sort: 10, 7, 6.

① ③ ②.

O(nlogn) (Greedy) fails ✗ ₹38

3
1

6 − 1 = 5.

5 − 4 = 1

DP → ₹40

- → optimal SS
- → recursive eqns
- → overlapping SPs

* Why Greedy fails

- If capacity : W.

Obj	1	2.
Prof	2	W
Weight	1	W

① ②

P/W = 2 1.

1

W − 1. ₹2.

if W = 1mn,

Greedy will give ₹2.

* Substructure & Recursive Eqn.

1 2 3 4 ····· n.

2 choices. [include/ exclude]

∴ Total Sol^n = 2^n (Brute force).

Recursive Eqns

KS(i, w) = max { Pi + KS(i−1, W−Wi) // inc ; KS(i−1, W) // dec. ; 0 ; i=0 or W=0 ; KS(i−1, W) ; Wi > W [exclude] }

KS(i, w): i → item from 1 to i ; w → wt remain.

* Recursion Tree & Unique SubProblems

If all obj is wt 1 AU

KS(10,10) — 10 obj; 10 wt

KS(9,9) KS(9,10)

K(8,8) K(8,9) K(8,9) K(8,8)

K(7,7) K(7,8) K(7,8) K(7,9)

⇒ depth = O(n).

∴ T = O(2^n) // Complexity with Recursive App.

S = O(n)

∴ Lot of overlapping SP : DP.

∴ No. of Unique SP = O(n²)

⇒ O(n × W) (Worst case)

Table size → (n+1)(W+1)

K(1,1)

Q C = 6

	1	2	3
W	1	2	4
P	10	12	28

capacity 3, include both ob 1&2. max [22]. [∵ consider 1,2]

obj \ W	0	1	2	3	4	5	6
0		0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	12	22	22	22	22
3	0	10	12	22	28	38	40

→ Ans

→ with C=5, take 10, 28

- K(1,1) = { P1 + K(0,0) , K(0,1) ⇒ 10
- K(1,2) = { P1 + K(0,1) , K(0,2)
- K(1,3) = { P1 + K(0,2) , K(0,3)
- K(2,1) = { K(1,1) // deduct
- K(2,2) = { P2 + K(1,0) , K(1,2) ⇒ 12 + 10 , 10 ⇒ 22
- K(2,3) = // deduct = { P2 + K(1,1) , K(1,3)
- K(2,4) = 22
- K(3,1) = { K(2,1)
- K(3,4) = { P3 + K(2,0) , K(2,4)
- K(3,6) = { P3 + K(2,2) = 40 , K(2,6) = 22 → 40

Bottom up Manner. [Started from small to big]

T = O(n*W) , S = O(n*W)

Scanned with CamScanner

* but in cell, $w = 2^n$.
$T = O(n \cdot 2^n)$.

if, w is too big so for Brute force

$$T(n) = \min\{O(2^n), O(nW)\}$$

↳ depends upon W.
→ not Polynomial tym.

- We are stuck, if w very large.

* Subset Sum [Decision Problem]

- Is any subset which is equal to w

{6, 2, 3, 1} S = 5
↓
{2, 3}.

- Greedy → small/high (what?) ✗
- Brute force
↳ No of SS = 2^n ∴ $T = O(2^n)$

Recursive

$$SS(i, S) = \begin{cases} SS(i-1, S-a_i) \vee SS(i-1, S) & \text{include } i^{th} \text{ item, exclude} \\ \text{True} & i=0, S=0 \\ \text{False} & i=0, S \neq 0 \\ SS(i-1, S) & S < a_i \end{cases}$$

{1, 2, 3, 4}
10 - 4 = 6

S(10,10)
S(9,9) S(9,10)
S(8,8) S(8,9).
S(7,7) S(7,8)
:
n = depth
$T = O(2^n)$ (& overlapping subproblems).

∴ No of unique SPS = $O(n \times S)$ ← use of this.

* Example
{6, 3, 2, 1} S = 5
n = 4, S = 5
∴ No of P = 20.

Elem \ Sum	0	1	2	3	4	5
0	T	F	F	F	F	F
1	T	F	F	F	F	F
2	T	F	F	T	F	F
3	T	F	T	T	F	T
4	T	T	T	T	T	T

Row taking 1st elem

in 2 element Set, SS is not Poss with sum = 4.

S(4,2) = S(3,1), S(3,2)

Ans

S(2,1) = S(1,1).

S(2,3) = S(1,0) ∨ S(1,3) = T (T ∨ F)

S(2,4) = S(1,1) ∨ S(1,4).

S(3,2) = S(2,0) ∨ S(2,2)

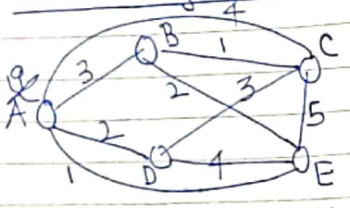
Scanned with CamScanner

$T = O(n * S)$

Using DP: depends on S.
if S is high → use Brute force.

∴ 01 KS / SubSum : np complete Problems.

* Travelling Salesman Problem



. 5 Villages.

- Visiting every Vertex exactly once, & coming back : Hamiltonian Cycle.
- Find the min cost HC.

[Visit every Vertex, with min distance covered].

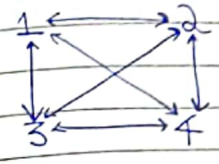
- **Brute Force :**

(A) B C D E (A).

(n−1)! Paths.

$O(n!) \Rightarrow \boxed{O(n^n)}$ → Very Bad. [Brute Time].

* Detecting Optimal Substructure



	1	2	3	4
1	0	1	2	3
2	1	0	4	2
3	1	2	0	5
4	3	4	1	0

Not Symm. [Costs diff]

$$T(1,\{2,3,4\}) = \min \begin{cases} (1,2) + T(2,\{3,4\}) \\ (1,3) + T(3,\{2,4\}) \\ (1,4) + T(4,\{2,3\}) \end{cases}$$

[start from 1, cover 2 3 4, & get to 1].

$$\Rightarrow T(2,\{3,4\}) = \min \begin{cases} (2,3) + T(3,\{4\}) \\ (2,4) + T(4,\{3\}) \end{cases}$$

$$T(3,\{2,4\}) = \min \begin{cases} (3,2) + T(2,\{4\}) \\ (3,4) + T(4,\{2\}) \end{cases}$$

$$T(4,\{2,3\}) = \begin{cases} (4,2) + T(2,\{3\}) \\ (4,3) + T(3,\{2\}) \end{cases}$$

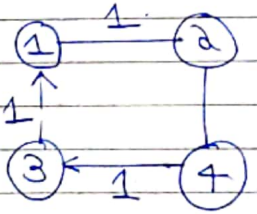
$$T(3,\{4\}) = \begin{cases} (3,4) + T(4,\{\}) \\ \end{cases}$$

→ from 4 to 1.. (4,1)

↳ Smallest Problem.

Final Ans = 5

Trace Back →



Scanned with CamScanner

* Bottom up DP

Recurs eqn → $T(i,S) = \min_{j \in S} \left[(i,j) + T(j, S-\{j\}) \right]$; $S \neq \phi$

Set of vertices not yet visit.

$T(i, \phi) = (i, 1)$; $S = \phi$

$V = \{1, 2, 3, 4\}$

Recursion tree →

$T(1,\{2,3,4\})$

$T(2,\{3,4\})$ $T(3,\{2,4\})$ $T(4,\{2,3\})$

$T(3,\{4\})$ $T(4,\{3\})$ $T(2,\{4\})$ $T(4,\{2\})$ $T(2,\{3\})$ $T(3,\{2\})$

$T(4,\{\phi\})$ $T(3,\phi)$ $T(4,\phi)$ $T(2,\phi)$ $T(3,\phi)$ $T(2,\phi)$

∴ Dynamic Programming Sub problems. → Have 12 unique SPs

H

At worst level : $i = (n-1)$ values.

S

$(1, (n-1))$.

$n-1$ SPs (each size $(n-1)$.

$(n-2) \times (n-1) SPs$ (each size $n-2$)

$(n-3)$ $(n-2)$ $(n-1)$ SPs.

$\therefore (n-1) + (n-1)^{n-2} + (n-1)^{n-2} + \ldots^{n-1} C_{n-4}$

$(n-1)a^{n-a} = $ No of unique SPs.

$\Rightarrow \Theta(a^n)$: No of unique SPs

∴ naive applying DP : Dynamic algo $O(n \cdot a^n)$

better (but still exponential).

CNO Paddy stone exists

dreams / such dreams also possible.

∴ NP complete.

Cube Size : $(n-1)a^{n-2}$. → 12 SPs @

$\rightarrow (4-1)a^2 = 12$

$\therefore \boxed{T = O(n^2 2^n)}$, $S = O(n-1)2^{n-2}$

$\boxed{S = O(n^2 2^n)}$ $O(n^2 2^n) \times O(n)$

NP complete. unique SP some extra work to solve a prob

* Floyd Warshall → Dynamic, Single Source SP.

- All pairs shortest Path Problem

(many node src).

Run Dijkstra on using Vertex X

$\therefore O(E \log V) \times V = O(VE \log V) \times O(V^3 \log V)$ dense Graph

∴ Dijkstra slow, & cant work with -ve

x Bellman ford : $O(VE) \cdot V = O(V^2 E) = O(V^4)$

* Optimal Substructure

$V = \{1, 2, 3, 4 \cdots n\}$

$i, j \in V$.

- d_{ij}^{K} : Shortest dist b/w $i \to j$ which includes only 1 to K vertices.

$\Rightarrow$ P.

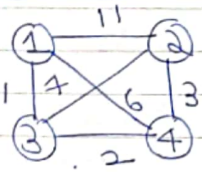
$d_{ij}^{(K-1)}$ (without using K.) $\quad d_{ik}^{K-1} + d_{kj}^{K-1}$ $\quad K=0 \Rightarrow W_{ij}$.

$i\ 1\ 2\ 3\ K\ J\ m\ j \cdots$ | Bottom Up evaluation

FW: -ve, & directed ✓

(-ve edge cycle, it won't detect will give wrong ans).

*



dist with no vertex on way

$$D^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 11 & 1 & 6 \\ 11 & 0 & 7 & 3 \\ 1 & 7 & 0 & 2 \\ 6 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

← Symmetric ∵ undirected Graph.

path allowed to pass thru 1

$$D^1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 11 & 1 & 6 \\ 11 & 0 & 7 & 3 \\ 1 & 7 & 0 & 2 \\ 6 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

- $(4,2) = (3,\ 4-1+1-2)$ $\quad 6 \quad 11$
  $= 3$.
- $(4,3) = (2,\ 4-1+1-3)$ $\quad 6 \quad 1$
  $= (2)$

D^1.

allow 1 & 2 in Path

$$D^2 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 11 & 1 & 6 \\ 11 & 0 & 7 & 3 \\ 1 & 7 & 0 & 2 \\ 6 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

- $(1,3) = (1,\ 1-2+2-3)$ $\quad 11 \quad 7$
- $(1,4) = (6,\ 1-2+2-$ $\quad 11 \quad 3$
- $(3,4) = (2,\ 3-2+2-4$. $\quad 7 \quad 3$.

D^2.

allow 1, 2, 3 in Path.

path from 1 to 3 incl 1

$$D^3 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 8 & 1 & 3 \\ 8 & 0 & 7 & 3 \\ 1 & 7 & 0 & 2 \\ 3 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

- $(1,2) = (11,\ 1-3+3$ $\quad 1 \quad 7$
- $(1,4) = (6,\ 1-3+3-$ $\quad 1 \quad 1$
- $(2,4) = (3,\ 2-3+3-4)$. $\quad 7 \quad 1$

allow 1, 2, 3, 4

$$D^4 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 6 & 1 & 3 \\ 6 & 0 & 5 & 3 \\ 1 & 5 & 0 & 2 \\ 3 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

- $(1,2) = (8,\ 1-4+4$ $\quad 3$
- $(1,3) = (1$
- $(2,3) = (7,\ 2-4+4-3)$ $\quad 3 \quad 2$

↓ Len of SP b/w every pair.

Scanned with CamScanner

Scanned with CamScanner

* Time complexity & Space of Bottom up DP

D¹ ... D⁴. $n \rightarrow$ Vertices

$\therefore n^2 \times 4 \quad \therefore n^2 \times n$

$\therefore n^3$ ~~traversal~~ SPs/~~length~~ calls.

each func^n: O(1) time.

$\therefore$ $T = O(n^3)$

$\Rightarrow$ Space: Max Space: we need only 2 Mats

$\Rightarrow 2 \times n^2$.

$S = O(n^2)$ (Optimized, reuse the matrices).

* Bellman Ford

has the ability to process -ve edges

G $\longrightarrow$ BF $\begin{cases} \rightarrow \text{No } (-\infty \text{ cycle}) \\ \rightarrow \text{Yes (SP)}. \end{cases}$

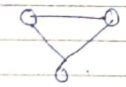
$\therefore$ BF handles -ve edge Graphs

Drawback: Slower than Dijkstra.

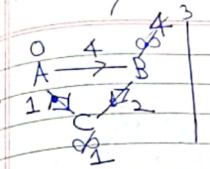
· in real time, -ve edges less.

* If G $\longrightarrow$ n vertices.

SP will never have more than n-1 edges.



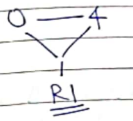
SP can contain 2 edges at most.



If n nodes, relax very edge (n-1) times.

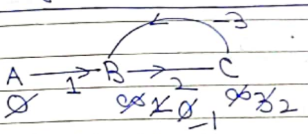
· (if relaxing 1 time, the SP contains at most 1 edge)

$\therefore$ relax 2 times.



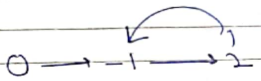
2 relaxation

· do 1 more relaxation, if value changes, $\longrightarrow$ -ve cycle. ✓



AB, BC, CAB

BF runs 2 times.



: after 2 ro

After 3rd round: Value changes.

$\therefore$ -ve edge cycle

$\therefore (V-1)+1 \quad \therefore$ V times [Array not O(1)

$\therefore O(VE)$ relaxation.

$\therefore$ $T = O(VE)$

* Bellman Ford Example

∴ 4 relaxn to stabilise. [SP].

- relax with prev G.
- pick any order of edges.

→ R1 of relaxn: len of SP (atmost 1 edge)
→ R2 : len of SP (atmost 2 edges).

$T = O(VE)$

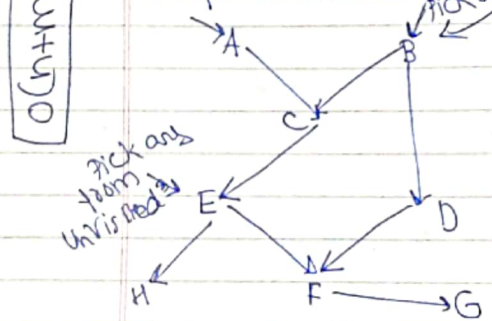
* Bellman Ford Algo

Initialize: O(V)
Outer Loop runs V-1 times, inner loop for E times.
∴ $T = O(VE)$

→ capacity to find out if -ve edge cycle exists.

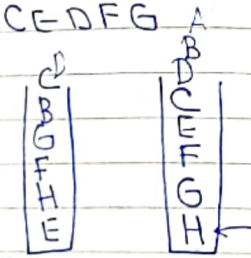
* Topological Sort

O(V+E)



Build System, Pack System (depends)

- ABCEDFG



∴ ABDCEFGH.

* NP Completeness

NP (not solved in polynomial time) : $O(n^K)$
↳ can't solve it, means has solved it till now in poly time. (Prove it).
[NP completeness].

* Types of algorithms.

Algo → complete (halts)

Prob

Decidable probs (algo exists)

[HP of TM] Undecidable (no algo exists)

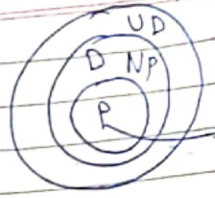
Tractable
→ if there exists at least one polynomial bound algo $O(n^K)$
(K any const)
→ $n, n^2, \ldots n^{1000}$
→ Best algo / only
- if n^{1000}... there definitely exists, other optimize algo to bring down n^2/n^3...

e.g. real time S/W solutions.

Intractable
→ if not tractable
$O(n!), O(n^n), O(c^n)$
- like TSP: $O(n^2 \cdot 2^n)$
- Slow / worst algo
→ Such Prob exists in real world, we can give approx results in poly time (heuristic) like, Path in TSP.

Scanned with CamScanner

Problems



NP → TSP: as of now, polyn time dne. In future, maybe polyn exist or someone will prove, poly time dne.

* Some Examples

Many Problems look tractable at Surface, but actually hard.

- → SP → DijKstra $O(E \lg V)$ [Easy]
- → longest Path → $O(n!)$ [Untractable]
- → Euler tour → $O(E)$ (E: MG).
- → min cost Hamiltonian cycle → $O(n^2 \cdot 2^n)$

- 2CNF [Conjuctive NF] : Easy
  - $(x_1 \vee x_2) \wedge (x_3 \vee \overline{x_4}) \cdots$
- 3CNF : Hard.
  - $(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee \overline{x_2})$

* Optimisation & Decision Problems

→ Proof that no Poly time exits till now.

Q. TSP [find min cost Hamiltonian Cycle]

Q. 0 1 KS

Q. LCS if both Strings len = n 2^n
$\therefore O(n^n \times 2^n) = 2^{2n}$.

(Optim Probl)

Q. TSP → easy / hard?

Is there any SP covering all vertices of length atmost K? (: minimin)

(Yes/No)

if you cant solve this, which is easy, dont trik to solve TSP (hard)

Decision Problems → Yes/No Ans

[Work on this b4 optimizn Problems]

Q. Is there any Solution whose Profit is atleast K? (maximin)
→ Decision Problem of 0/1 KS

Q. Is there any subSequence whose length is atleast K? (maximin)

∴ Optimizn Problem —conv.→ decision Problem.

Scanned with CamScanner

* We use only Decision Problems in NP Completeness theory.

• Optimizⁿ Problem (1) —convert→ Decis Prob (2) [Y / N]

if (1) can be solved, (2) easily solved.
if (1) easy, (2) easy.

if opt: Problem is easy/nK	→	DP is easy/ O(nK)

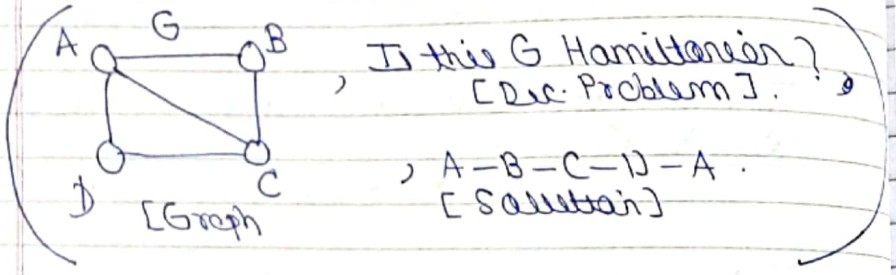
like TSP: 10 atmost 15 : Yes.

if DP is hard/ intractable	→	opt: Problem is hard

$p \to q$, $\neg q \to \neg p$

Q Proof DP is hard : NP Completeness theory.

* Verification Algorithms



, Is this G Hamiltonian? [Dec. Problem].

, A–B–C–D–A. [Solution]

Q Simply Verify? Is Soln Hamilt?

→ Algo is Verification Algo.

∴ A–B–C–D–A exists.

(1) Check if all V. present?
(2) Check if all the edges (in solution) are present or not?

• Verification Algo.

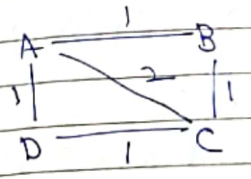
* Why VA?

OP —— DP, S

if in future, someone solves OP in P(nK) & gives Soln (S) – Verify DP with S. if VA = O(nK), DP is easy.

if we can have polyn Time VA → done.

Q



(Does TSP exists, A B C D A at most 5)

VA : Vert. ✓ edges ✓ $1 \times 4 = 4 \leq 5$ ✓

∴ Verified Yes → in O(nK).

∴ TSP has a Polyn time VA.

(∵ If OP → Soln (O(nK)), VA → O(nK) ✓)

→ Intuited ans

Scanned with CamScanner

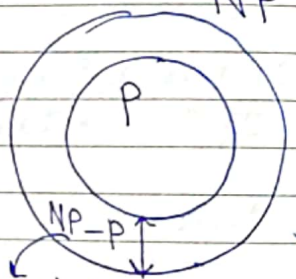
only [illegible]

* P-NP Introduction

P: Set of decision Problems for which there is Polynomial time Algorithm to solve them

NP: Set of decision Problems whose solution can be verified in Polynomial time / Polyn time Verifier Algorithm.

P	NP
→ G, MST atmost 10 [Prims → O(nK)] VerifA → O(nK)	→ TSP. [No Poly time Soln yet] VA → Poly time.
→ fKS, Is there any Soln whose Profit is atleast 10? [fKS → O(nlogn). VA → O(nK).	→ 0/1 KS. [No Poly time soln yet] VA → O(nK).
→ (Greedy & DnC all) ↓ • Poly time Soln. • Poly time VA.	(if in future, someone solves in O(nK), it will become P)



intractable.

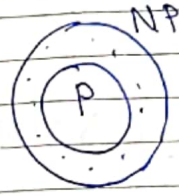
[∵ 1971 till today, no Poly exists] has proved that no Poly time will exist in future.

P ⊂ NP

* if all (NP-P) Problems can be solved in Polynomial time, then P ≈ NP.
* for showing P ⊂ NP, you need to prove that atleast 1 Problem of (P-NP) Set, can't even be solved in Poly time.

∴ Relⁿ b/w P & NP not known yet

* Polynomial time reduction Algorithms



P — quickly Solved or easily Solved.
NP — quickly Verified or easily Verified

∝ to declare P=NP, come up with a single Problem in (NP-P), (harder than all), if that can be solved in Poly time, all can be solved

* A Problem 'A' is said to be polynomial time reducible to a Problem 'B' if: A → B
  (i) If instance 'α' of A can be transformed to some instance 'β' of B in Poly time
  (ii) Ans to α is Yes, if & only if answer to β is Yes

Polynomial time reduction

A → B
α → β
Y Y

$A \xrightarrow{O(nK)} B$

$\alpha \longrightarrow \beta \quad O(nK)$

↓ $O(n^K)$ Algo to solve β.

1) Convert to β, solve β, Ans to β is Ans to α.

① ↓ 'B' is easy, then 'A' is also easy.
↓ 'B' is in P, then 'A' is in P.

$A \xrightarrow{O(n^K)} B$ ∴ B = hard

A: Not in P (No poly time exists)

[Proof].

② If 'A' is not in P (hard), then definitely B is not in P.

Q A: Given n Boolean Var. with values $X_1, X_2 \cdots X_n$, does atleast 1 variable is True.

B: Given n integers $i_1 \cdots i_n$ is $\max(i_1, i_2, i_3 \cdots i_n) > 0$.

Both should be Decision Problem & solve = $O(nK)$

eg $n = 4$
(TFFT)
A → True

B → True.
(-30, 10, 0, 2).

B → O(n) solved.

* $A \longrightarrow B$

Reduction algo: (TFFT) → (1001)
O(n)

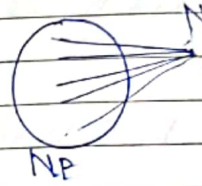
$A \xrightarrow{O(n)} B$

↓
Solved: O(n).

∴ A → Solved = O(n) + O(n)

* NP Hard & NP Complete

Problem A



↓ A → easy, all NP = easy

∴ NP = P

* NP-Hard: If every Problem in NP can be polynomial time reducible to a problem 'A', then A is called NP-Hard.

→ If 'A' could be solved in Polynomial time, then every problem in NP is 'P'.

$P = NP$

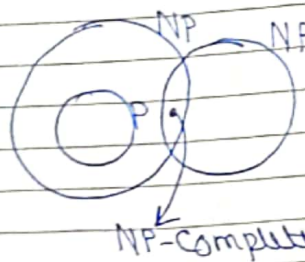
↳ till now no one could find sol^n to any NP Hard Problem.

Scanned with CamScanner

* If 'A' is NP & NP Hard ⟶ NP Complete
[Problem is same as NP]

* NP Complete : If it is 'NP' and 'NP Hard'.

* We believe (not Proved)
relation.



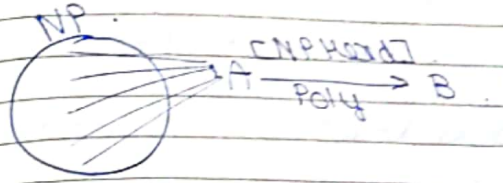
⟶ Take NPC problem, give a Polytime to it
∴ all NP Problem can also be Solved.
∴ P = NP

⟶ as proved, NPC can not be Solved.
[Not in P]
∴ P ⊂ NP
(Nobody could do this)

* If NP Hard / NP Complete Problem is Solved in Polynomial time, then ~~if~~ problem in NP can be solved in Pt.
then P = NP.

* NP Problem or NP Complete Problem is proven to be not Solvable in Polynomial time
(?) NP then P ≠ NP ✓

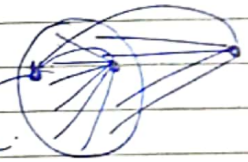
* Status of NP is unknown
[We just know it can be Verified in Poly. time, nothing abt Solvable time]

* 

if B is NP ⟶ NP Complete

* If A is NP-Hard and A ⟶ B (Poly time), then
B becomes NP-Hard.
⟶ & if B is already in NP, & A ⟶ B (Poly) then
B is NP Complete.

* Well Known NP Complete Problems



- ~~As of now~~, Poly time done
- in either way, if you Solve : P = NP.

1. Circuit-Satisfiability Problem

And comb of i/ps

Proof: if this could be Solved, or it can be solved

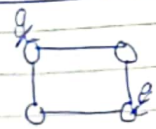
Scanned with CamScanner

2. Satisfiability Prob: $(x+y+z)\cdot(\bar{x}+\bar{y})=1$?
   find x, y, z ...
   CSAT $\xrightarrow[\text{time.}]{\text{Poly}}$ SAT.

3. 3NF-SAT: $(x+y+\bar{z})\cdot(\bar{y}+z+W)=1$?

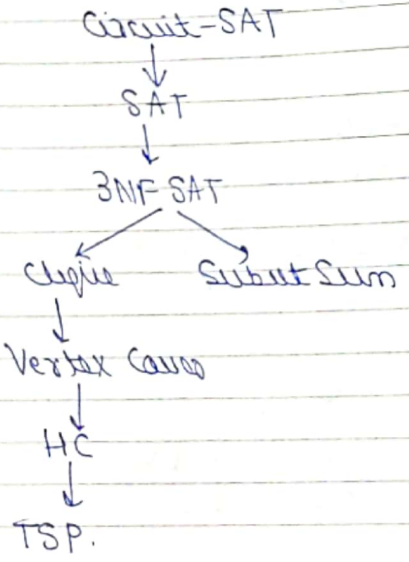
4. clique: what is the max K_n subGraph of a graph?

5. Vertex Cover: How many min Vertices to choose to cover $\forall$ edge?
   [Verified in Poly time].



6. Subset Sum Prob: $O(N \cdot S)$
   [Verified in Poly time].

7. Ham Cycle ⟶ TSP.

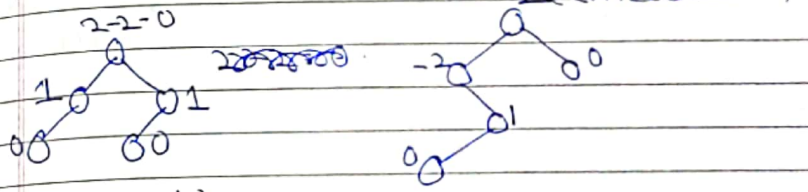


* AVL Tree [Abdul Bari]

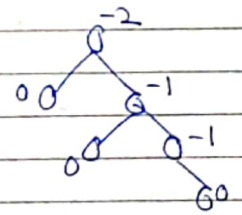
- BST ⟶ useful for Searching.
- ht. depends upon order of insertion (not under control)
  [log n, n]
- for 3 keys: 3! orders are possible.
  [diff shapes of BST].
- * Rotation based on 3 Nodes only.

* AVL Trees

- height balanced BST.
  bf = ht of LS − ht of RS
  bf = {−1, 0, 1}
  $\boxed{|bf| = |ht - hr| \leq 1}$

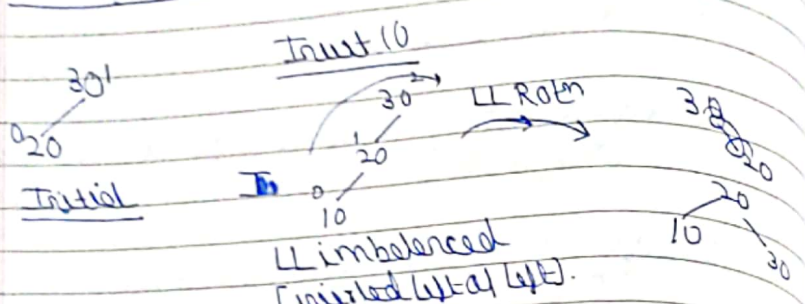


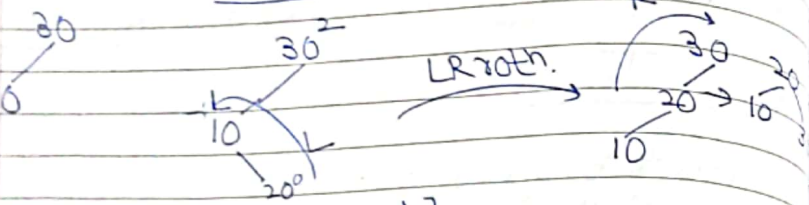
(balanced)



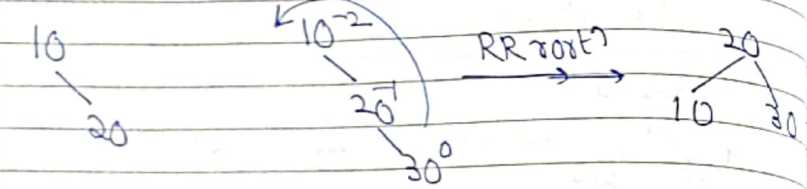
Scanned with CamScanner

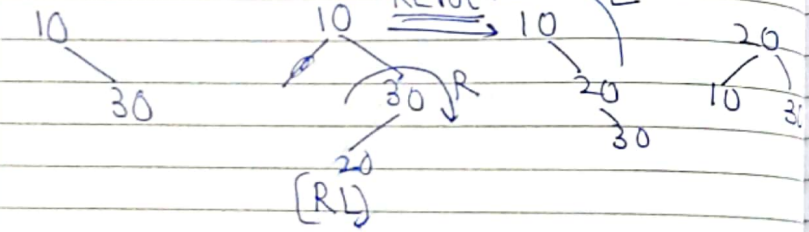
* Rotations.

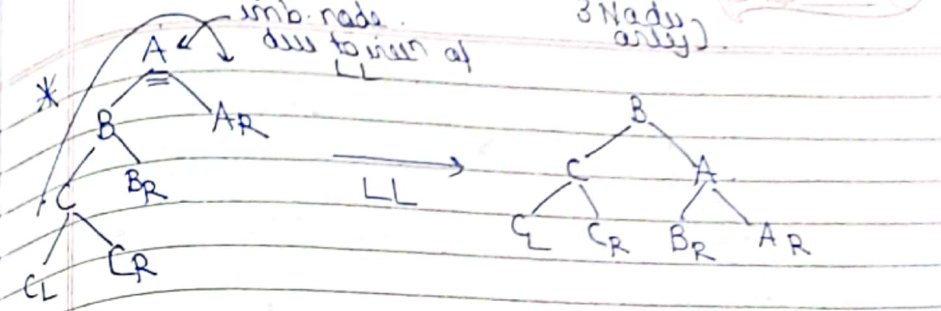


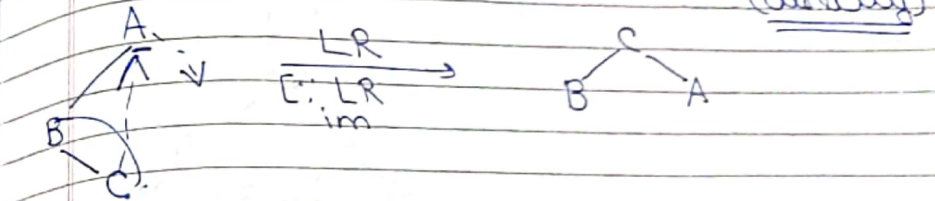


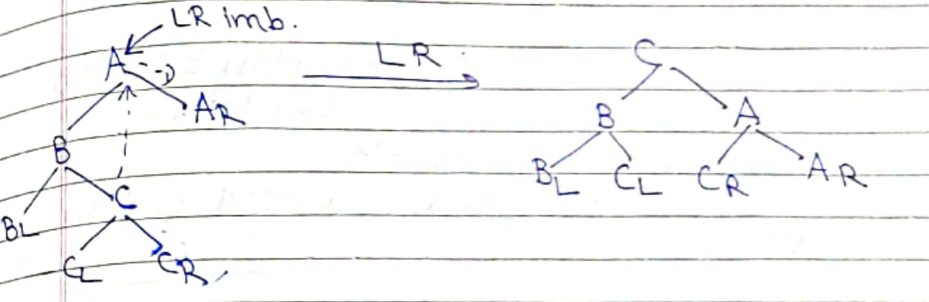
(double rotation) not 2 rotns



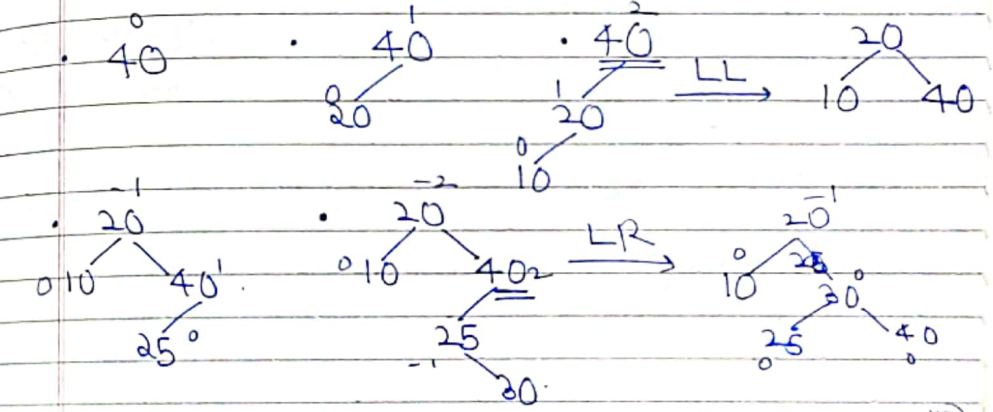






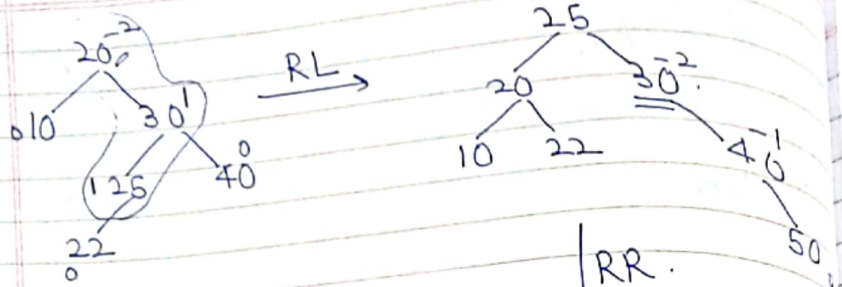


* Keys → 40, 20, 10, 25, 30, 22, 50

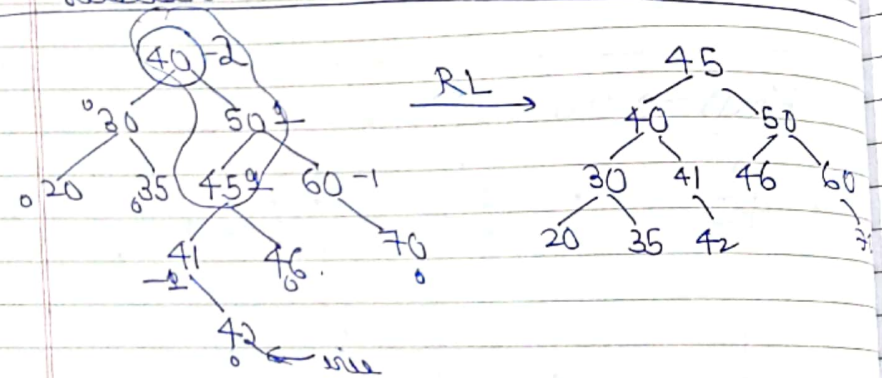


Scanned with CamScanner

[only 3 Nodes]



→ You should never get −3 ...
when any node gets inbalanced, perform rotation.



[always logn height]

atmost = 1.44 log n

Almost Complete BT → Red Black Tree

* Strassen's Matrix Mult.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}_{2\times2} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2\times2} = C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}_{2\times2}$$

$$c_{ij} = \sum_{k=1}^{n} A_{ik} * B_{kj}$$

```
for(i=0; i<n; i++)
{
  for(j=0; j<n; j++)
  {
    c[i,j] = 0;
    for(k=0; k<n; k++)
    {
      c[i,j] += A[i,k] * B[k,j];
    }
  }
}
```

$T = O(n^3)$

* DnC

Small Problem : mult. 2×2
if 4×4 / 8×8 : apply DnC technique.

just form:
$c_{11} = a_{11}b_{11} + a_{12}b_{12}$
$c_{12} = a_{11}b_{12} + a_{12}b_{22}$ [1 row, 2nd co
$c_{21} = a_{21}b_{11} + a_{22}b_{21}$
$c_{22} = a_{21}b_{12} + a_{22}b_{22}$

⇒ O(1) : Small Prob

Scanned with CamScanner